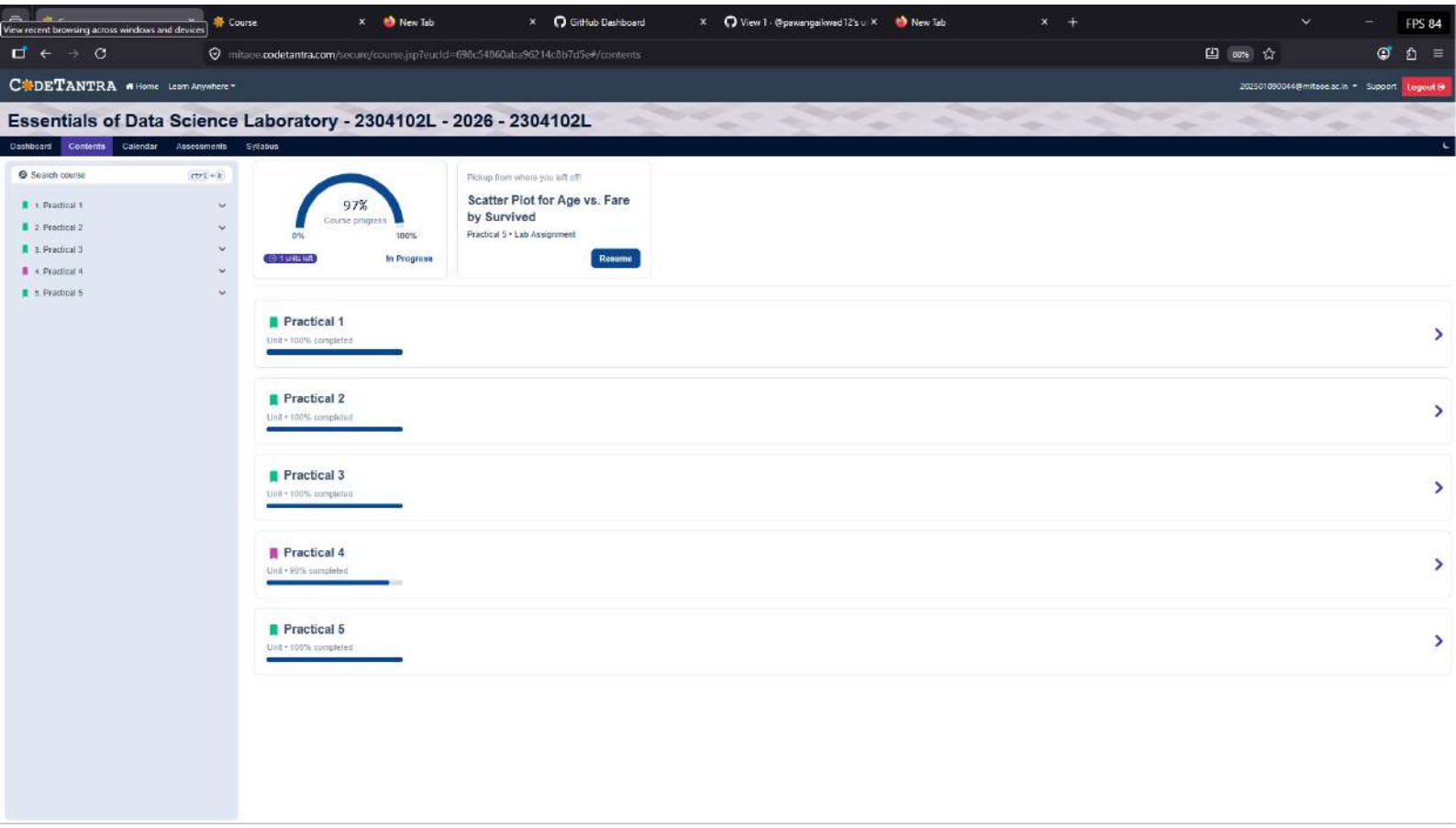
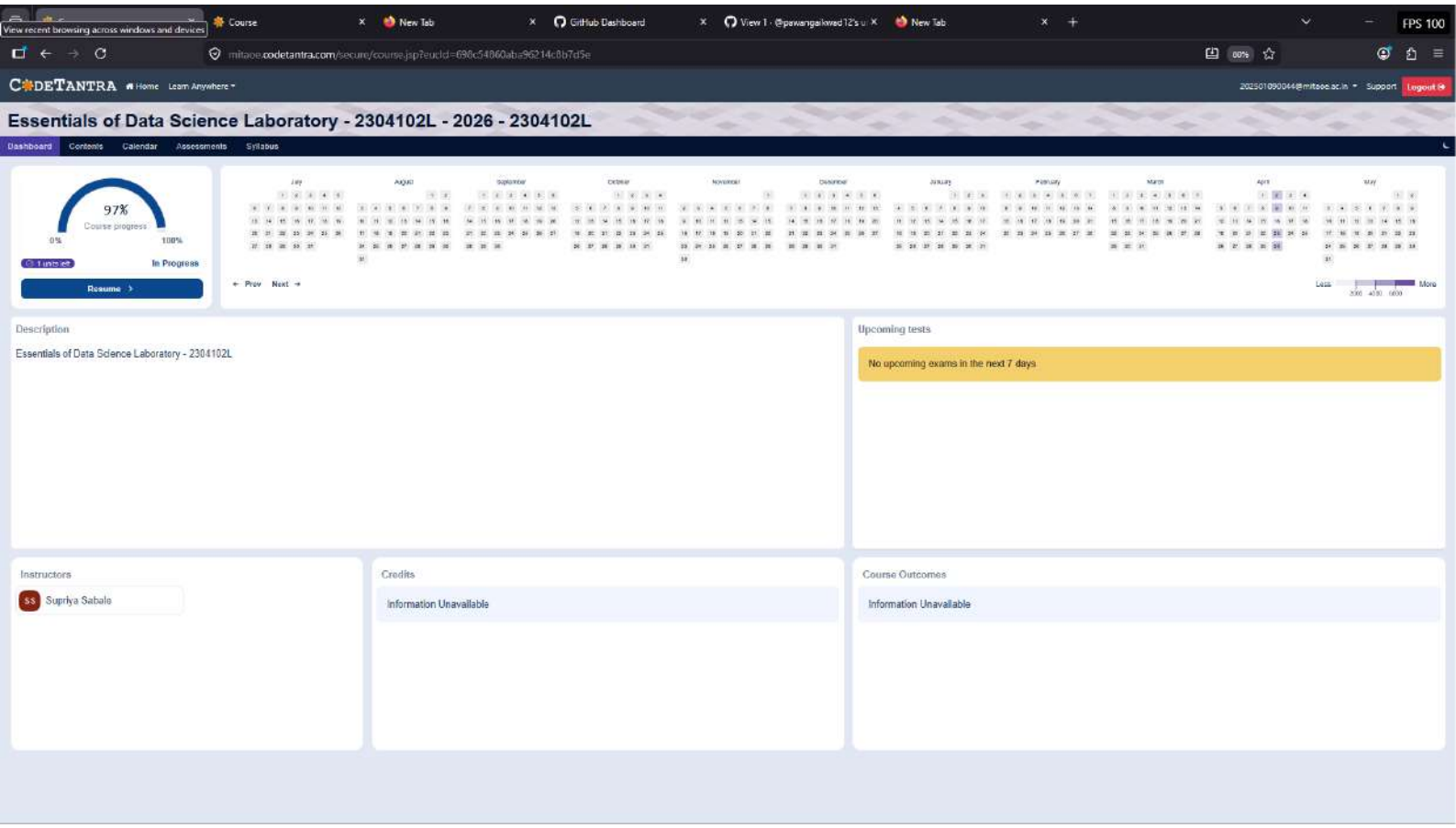

Course
New Tab
GitHub Dashboard
New Tab
FPS 84
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54060aba96214c8b7d5e#/contents
CODETANTRA
Home
Learn Anywhere
Support
Logout
Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L
Dashboard
Contents
Calendar
Assessments
Syllabus
Search course
1. Practical 1
2. Practical 2
3. Practical 3
4. Practical 4
5. Practical 5
97%
Course progress
0%
100%
In Progress
Pickup from where you left off!
Scatter Plot for Age vs. Fare by Survived
Practical 5 • Lab Assignment
Resume
Practical 1
Unit • 100% completed
Practical 2
Unit • 100% completed
Practical 3
Unit • 100% completed
Practical 4
Unit • 90% completed
Practical 5
Unit • 100% completed


Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
New Tab
FPS 100
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e
CODETANTRA
Home
Learn Anywhere
Support
Logout
Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L
Dashboard
Contents
Calendar
Assessments
Syllabus
97%
Course progress
0%
100%
In Progress
Resume
Prev
Next
Description
Essentials of Data Science Laboratory - 2304102L
Upcoming tests
No upcoming exams in the next 7 days
Instructors
Supriya Sabale
Credits
Information Unavailable
Course Outcomes
Information Unavailable

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 53

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e2a5387c0b912d860eba29

CODETANTRA Home Learn Anywhere 2025109044@mitaoe.ac.in Support Logout

5.2.11. Scatter Plot for Age vs. Fare by Survived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**), **Blue** for passengers who survived (**Survived = 1**).
3. Set the **title** of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Scatter Plot for Age vs. Fare by Survived
# Write your code here for Scatter Plot for Age vs. Fare by Survived
colors = data['Survived'].map({0:'red',1:'blue'})
plt.scatter(data['Age'],data['Fare'], c=colors)
plt.title('Age vs. Fare by Survival')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

Average time: 0.465 s (465.00 ms) | Minimum time: 0.465 s (465.00 ms) | 1 out of 1 shown test case(s) passed

Test case 1 (465 ms) Debug

Expected output | Actual output

Terminal | Test cases

Prev | Reset | Submit

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 26

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e2a2977c0b912d860eb621

CODETANTRA Home Learn Anywhere 2025109044@mitaoe.ac.in Support Logout

5.2.10. Scatter Plot for Age vs. Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to "**Age vs. Fare**".
3. Label the x-axis as '**Age**' and the y-axis as '**Fare**'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Fare by Pclass

# Write your code here for Box Plot for Fare by Pclass
plt.scatter(data['Age'],data['Fare'])
plt.title('Age vs. Fare')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

Average time: 0.514 s (514.00 ms) Minimum time: 0.514 s (514.00 ms) 1 out of 1 shown test case(s) passed

Test case 1 514 ms Debug

Expected output

Actual output

Terminal Test cases

‹ Prev Reset Submit Next ›

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 34

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e2a1727c0b912d860eb39b

CODETANTRA Home Learn Anywhere 20250109044@mitaoe.ac.in Support Logout

5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the Pclass column to group the data for the boxplot.
2. Set the title of the plot to "Fare by Pclass".
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as "Pclass" and the y-axis as "Fare".

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

BoxPlotF... Submit

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16
17 # Write your code here for Box Plot for Fare by Pclass
18 import pandas as pd
19 import matplotlib.pyplot as plt
20
21 # Load the Titanic dataset
22 data = pd.read_csv('Titanic-Dataset.csv')
23
24 # Data Cleaning
25 data['Age'].fillna(data['Age'].median(), inplace=True)
26 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
27 data.drop('Cabin', axis=1, inplace=True)
28
29 # Convert categorical features to numeric
30 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
31 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
32
33 # Box Plot: Fare by Pclass
34 plt.figure(figsize=(8, 6))
```

Average time: 0.539 s (539.00 ms) | Maximum time: 0.539 s (539.00 ms) | 1 out of 1 shown test case(s) passed

Test case 1 Debug

Expected output

Actual output

Terminal | Test cases

‹ Prev | Reset | Submit | Next ›

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 114

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e29db57c0b912d860eab45

CODETANTRA Home Learn Anywhere 2025109044@mitaoe.ac.in Support Logout

5.2.8. Box Plot for Age by Survived

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

1. Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived)
2. Set the title of the plot to "**Age by Survival**".
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as "**Survived**" and the y-axis as "**Age**".

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

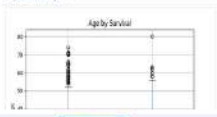
BoxPlotF... Submit

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Box Plot for Age by Survived
plt.figure(figsize=(8, 6))
data.boxplot(column='Age', by='Survived')
```

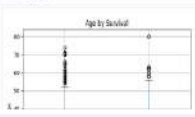
Average time: 0.414 s (414.00 ms) Maximum time: 0.414 s (414.00 ms) 1 out of 1 shown test case(s) passed

Test case 1 414 ms Debug

Expected output



Actual output



Terminal Test cases

‹ Prev Reset Submit Next ›

Course | Course | New Tab | GitHub Dashboard | View 1 · @pewangaikwad12's u | New Tab | FPS 85

mitaoe.codetantra.com/secure/course.jsp?eudid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e2992e7c0b912d860ea071

CODETANTRA Home Learn Anywhere 20250109034@mitaoe.ac.in Support Logout

5.2.7. Box plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below.

Passenger id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

BoxPlotF... Submit

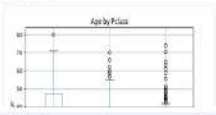
```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 # Write your code here for Box Plot for Age by Pclass
16
17 import pandas as pd
18 import matplotlib.pyplot as plt
19
20 # Load the Titanic dataset
21 data = pd.read_csv('Titanic-Dataset.csv')
22
23 # Data Cleaning
24 data['Age'].fillna(data['Age'].median(), inplace=True)
25 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
26 data.drop('Cabin', axis=1, inplace=True)
27
28 # Convert categorical features to numeric
29 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
30 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
31
32 # Box Plot for Age by Pclass
33 plt.figure(figsize=(8, 6))
34 data.boxplot(column='Age', by='Pclass')
```

Average time 0.557 s 557.00 ms | Maximum time 0.557 s 557.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 Debug

Expected output

Actual output



Terminal Test cases

Prev Reset Submit Next

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawargaikwad12's u | New Tab

mitaoe.codetantra.com/secure/courseJsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e297bf7c0b912d860e9c81

CODETANTRA Home Learn Anywhere 20250109044@mitaoe.ac.in Support Logout

5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.
The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using **pd.get_dummies()**), plot the survival count based on the **Embarked_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title **"Survival by Embarked "** to the chart.
4. Label the x-axis as **'Embarked'** and the y-axis as **'Count'**.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as **'Survived'** and **'Not Survived'**).

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cushing, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

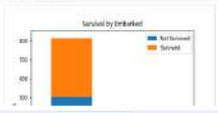
BarPlotOf... Submit

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Embarked
# Write your code here for Bar Plot for Survival by Embarked
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Embarked
```

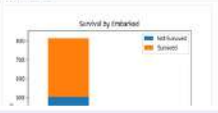
Average time 0.624 s 624.00 ms | Maximum time 0.624 s 624.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 624 ms Debug

Expected output



Actual output



Terminal Test cases

Prev Reset Submit Next

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's p | New Tab | FPS 102

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/67e291be7c0b912d860e876f

CODETANTRA Home Learn Anywhere 20250109044@mitaoe.ac.in Support Logout

5.2.5. Bar Plot for Survival by Pclass

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (**Pclass**), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **Pclass** column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using **value_counts()**
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title "**Survival by Pclass**" to the chart.
4. Label the x-axis as '**Pclass**' and the y-axis as '**Count**'.
5. The legend should indicate '**Not Survived**' and '**Survived**'.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note:

- Refer to the visible test case for better reference.
- Ensure you use the **groupby()** function with **value_counts()** to count the survivors and non-survivors for each **Pclass**.
- Do not manually use **size()** or **unstack()** without **value_counts()**. Use the **value_counts()** method for counting survival status directly.

Sample Test Cases +

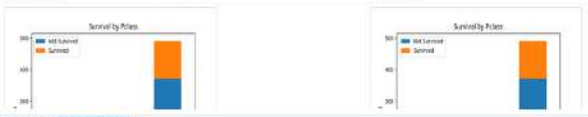
Explorer | BarPlotOf... | Submit | Debugger

```python
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Pclass
survival_by_class = data.groupby('Pclass')['Survived'].value_counts().unstack().fillna(0)
survival_by_class.columns = ['Not Survived', 'Survived']
survival_by_class.plot(kind='bar', stacked=True)
plt.title('Survival by Pclass')
plt.xlabel('Pclass')
plt.ylabel('Count')
plt.legend(title=None)
plt.show()
```

Average time 0.586 s 506.00 ms | Maximum time 0.586 s 586.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 506 ms Debug

Expected output | Actual output



Terminal | Test cases

‹ Prev | Reset | Submit | Next ›

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 23

mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d6b/690c54860aba96214c8b7d6d/67e28fb27c0b912d860e7db4

CODETANTRA Home Learn Anywhere 202501080044@mitaoe.ac.in Support Logout

5.2.4. Bar Plot for Survival by Gender

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender. In the Titanic dataset. The chart should display the following specifications:

1. Group the data by the 'Sex' column, then use the **value_counts()** function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Gender"** to the chart.
4. Label the x-axis as 'Gender' and the y-axis as 'Count'.
5. The legend should indicate 'Not Survived' and 'Survived'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

BarPlotOf... Submit

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Bar Plot for Survival by Gender
survival_by_gender = data.groupby('Sex')['Survived'].value_counts().unstack().fillna(0)
survival_by_gender.columns = ['Not Survived', 'Survived']
survival_by_gender.index = ['0', '1']
survival_by_gender.plot(kind='bar', stacked=True)
plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(title=None)
plt.show()
```

Average time 0.632 s (632.00 ms) Maximum time 0.632 s (632.00 ms) 1 out of 1 shown test case(s) passed

Test case 1 632 ms Debug

Expected output

Actual output

Terminal Test cases

Prev Reset Submit Next

View recent browsing across windows and devices

Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab | FPS 77

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6d/698c54860aba96214c8b7d6d/67e28d6e7c0b912d860e75ad

CODETANTRA Home Learn Anywhere 2025109044@mitaoe.ac.in Support Logout

5.2.3. Bar plot of survival rate of passengers

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

1. Use the '**Survived**' column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to 'bar'.
3. Add the title "**Survival Count**" to the chart.
4. Label the x-axis as '**Survived**' and the y-axis as 'Count'.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

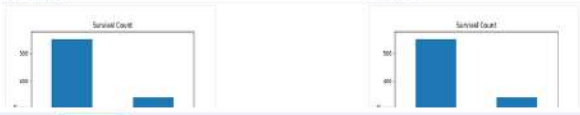
BarPlotOf... Submit

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 survival_counts=data['Survived'].value_counts()
16 survival_counts.plot(kind='bar')
17 plt.title('Survival Count')
18 plt.xlabel('Survived')
19 plt.ylabel('Count')
20 plt.show()
21
```

Average time: 0.502 s (502.00 ms) | Maximum time: 0.502 s (502.00 ms) | 1 out of 1 shown test case(s) passed

Test case 1 Debug

Expected output | Actual output



Terminal | Test cases

‹ Prev | Reset | Submit | Next ›

View recent browsing across windows and devices

Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab

FPS 79

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6d/698c54860aba96214c8b7d6d/67e28a397c0b912d860e689c

CODETANTRA Home Learn Anywhere

2025109044@mitaoe.ac.in Support Logout

5.2.2. Histogram of passenger information of Titanic

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:

1. Use **30 bins** for the histogram.
2. Set the **edge color** of the bars to **black** (k).
3. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
4. Add the title **"Age Distribution"** to the histogram.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases +

Explorer

Histogra...

Submit

Debugger

```python
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
plt.hist(data['Age'],bins=30,edgecolor='k')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```

Average time: 0.474 s (474.00 ms) | Maximum time: 0.474 s (474.00 ms) | 1 out of 1 shown test case(s) passed

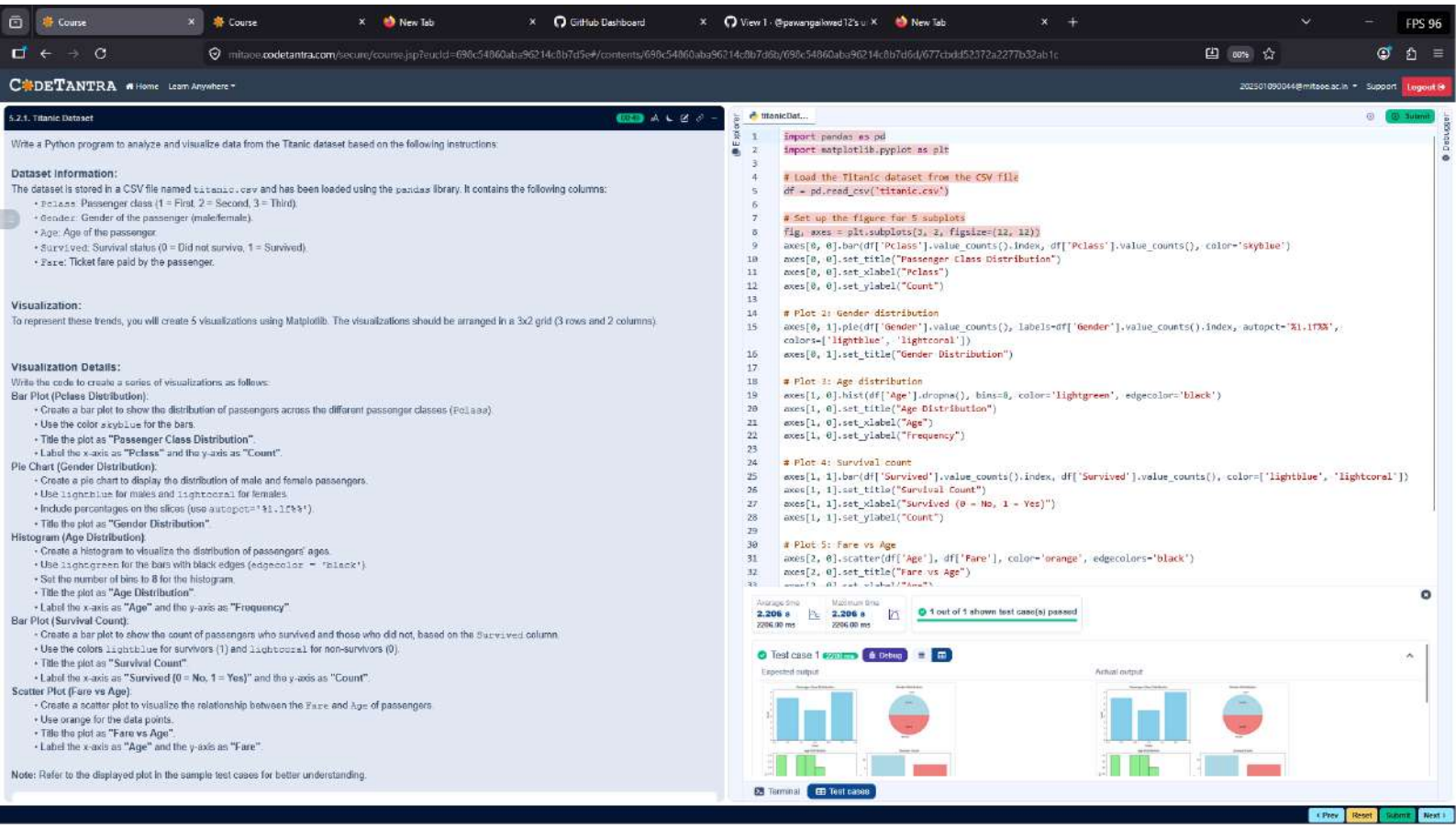
Test case 1 474 ms Debug

Expected output

Actual output

Terminal | Test cases

‹ Prev | Reset | Submit | Next ›


Course
Course
New Tab
GitHub Dashboard
New Tab
FPS 96
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6d/677cbdd52372a2277b32ab1c
CODETANTRA
Home
Learn Anywhere
Support
Logout
5.2.1. Titanic Dataset
Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:
Dataset Information:
The dataset is stored in a CSV file named titanic.csv and has been loaded using the pandas library. It contains the following columns:
• Pclass: Passenger class (1 = First, 2 = Second, 3 = Third).
• Gender: Gender of the passenger (male/female).
• Age: Age of the passenger.
• Survived: Survival status (0 = Did not survive, 1 = Survived).
• Fare: Ticket fare paid by the passenger.
Visualization:
To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).
Visualization Details:
Write the code to create a series of visualizations as follows:
Bar Plot (Pclass Distribution):
• Create a bar plot to show the distribution of passengers across the different passenger classes (Pclass).
• Use the color skyblue for the bars.
• Title the plot as "Passenger Class Distribution".
• Label the x-axis as "Pclass" and the y-axis as "Count".
Pie Chart (Gender Distribution):
• Create a pie chart to display the distribution of male and female passengers.
• Use lightblue for males and lightcoral for females.
• Include percentages on the slices (use autopct='%1.1f%%').
• Title the plot as "Gender Distribution".
Histogram (Age Distribution):
• Create a histogram to visualize the distribution of passengers' ages.
• Use lightgreen for the bars with black edges (edgecolor = 'black').
• Set the number of bins to 8 for the histogram.
• Title the plot as "Age Distribution".
• Label the x-axis as "Age" and the y-axis as "Frequency".
Bar Plot (Survival Count):
• Create a bar plot to show the count of passengers who survived and those who did not, based on the Survived column.
• Use the colors lightblue for survivors (1) and lightcoral for non-survivors (0).
• Title the plot as "Survival Count".
• Label the x-axis as "Survived (0 = No, 1 = Yes)" and the y-axis as "Count".
Scatter Plot (Fare vs Age):
• Create a scatter plot to visualize the relationship between the Fare and Age of passengers.
• Use orange for the data points.
• Title the plot as "Fare vs Age".
• Label the x-axis as "Age" and the y-axis as "Fare".
Note: Refer to the displayed plot in the sample test cases for better understanding.
titanicDat...
Submit
Explorer
Debugger
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')
# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
axes[0, 0].bar(df['Pclass'].value_counts().index, df['Pclass'].value_counts(), color='skyblue')
axes[0, 0].set_title("Passenger Class Distribution")
axes[0, 0].set_xlabel("Pclass")
axes[0, 0].set_ylabel("Count")
Plot 2: Gender distribution
axes[0, 1].pie(df['Gender'].value_counts(), labels=df['Gender'].value_counts().index, autopct='%1.1f%%',
colors=['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")
Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen', edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")
Plot 4: Survival count
axes[1, 1].bar(df['Survived'].value_counts().index, df['Survived'].value_counts(), color=['lightblue', 'lightcoral'])
axes[1, 1].set_title("Survival Count")
axes[1, 1].set_xlabel("Survived (0 = No, 1 = Yes)")
axes[1, 1].set_ylabel("Count")
Plot 5: Fare vs Age
axes[2, 0].scatter(df['Age'], df['Fare'], color='orange', edgecolors='black')
axes[2, 0].set_title("Fare vs Age")
Average time
2.206 s
2206.00 ms
Minimum time
2.206 s
2206.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
Actual output
Terminal
Test cases
Prev
Reset
Submit
Next

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6b/698c54860aba96214c8b7d6c/675a8d2c09f9fa022faa9dc0

CODETANTRA Home Learn Anywhere

2025010900044@mitaoe.ac.in Support Logout

5.1.1. Stacked Plot

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

Sample Test Cases

stackedpl...

Submit

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'],
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 8, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}
# create dataframe
df=pd.DataFrame(data)
plt.stackplot(df['Month'],df['City_A_Temperature'],df['City_B_Temperature'],df['City_C_Temperature'])
plt.xlabel('Month')
plt.ylabel('Temperature')
plt.title('Temperature Variation')
plt.show()
```

Average time: 0.438 s (438.00 ms)

Maximum time: 0.438 s (438.00 ms)

1 out of 1 shown test case(s) passed

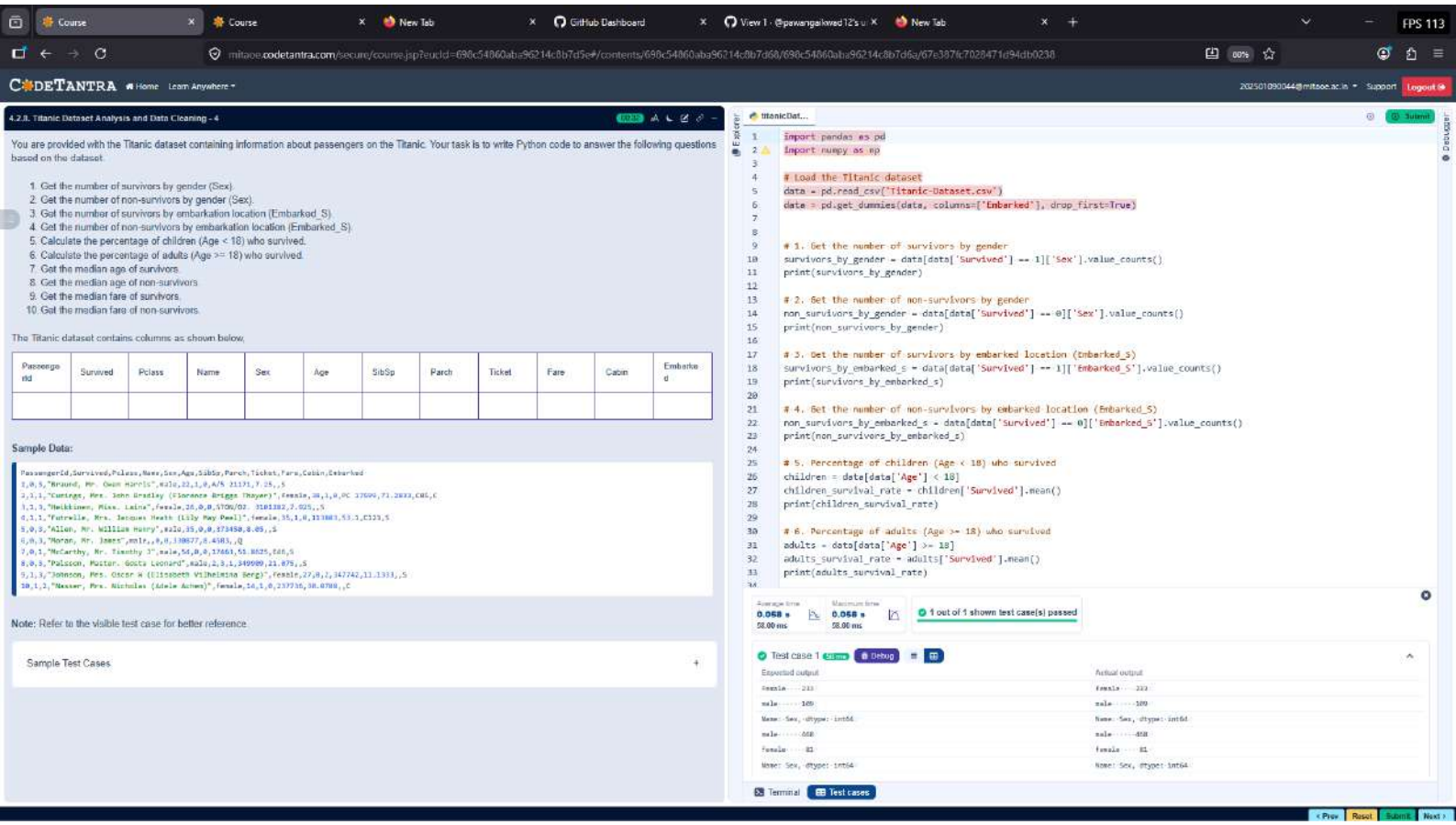
Test case 1 438 ms Debug

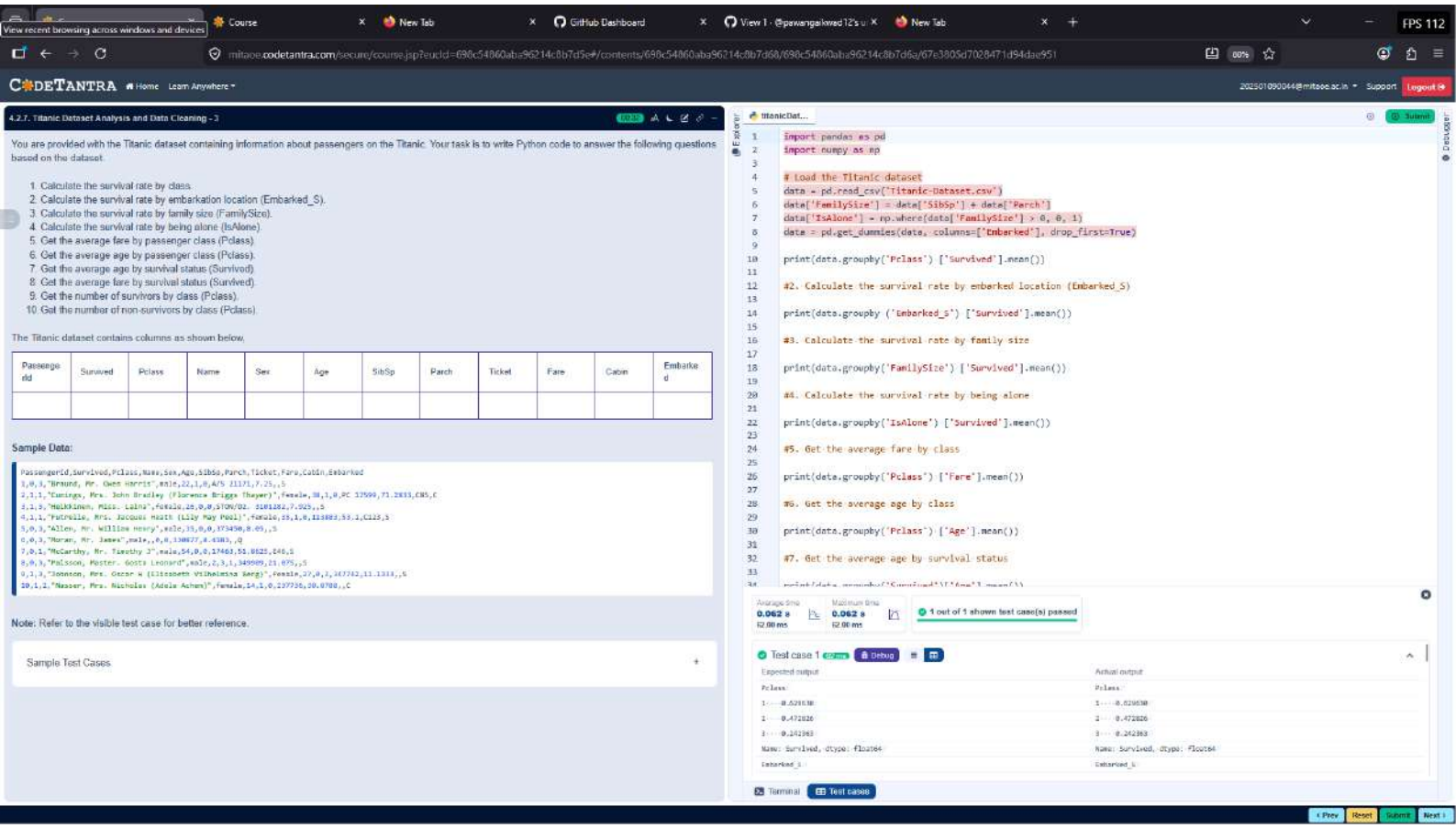
Expected output

Actual output

Terminal Test cases

‹ Prev Reset Submit Next ›


Course
Course
New Tab
GitHub Dashboard
New Tab
FPS 113
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d60/690c54860aba96214c8b7d6a/67e387fc7028471d94db0238
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.8. Titanic Dataset Analysis and Data Cleaning - 4
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.
The Titanic dataset contains columns as shown below.
PassengerId
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Sample Data:
Note: Refer to the visible test case for better reference.
Sample Test Cases
TitanicDat...
Submit
Explorer
Debugger
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# 1. Get the number of survivors by gender
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)
# 2. Get the number of non-survivors by gender
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
# 3. Get the number of survivors by embarked location (Embarked_S)
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
# 4. Get the number of non-survivors by embarked location (Embarked_S)
non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()
print(non_survivors_by_embarked_s)
# 5. Percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
# 6. Percentage of adults (Age >= 18) who survived
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
Average time
0.068 s
58.00 ms
Maximum time
0.068 s
58.00 ms
1 out of 1 shown test case(s) passed
Test case 1
58 ms
Debug
Expected output
female ... 233
male ... 109
Name: Sex, dtype: int64
male ... 468
female ... 81
Name: Sex, dtype: int64
Actual output
female ... 233
male ... 109
Name: Sex, dtype: int64
male ... 468
female ... 81
Name: Sex, dtype: int64
Terminal
Test cases
Prev
Reset
Submit
Next


Course
New Tab
GitHub Dashboard
New Tab
FPS 112
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e3805d7028471d94dae951
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.7. Titanic Dataset Analysis and Data Cleaning - 3
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).
The Titanic dataset contains columns as shown below.
PassengerId
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Sample Data:
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
Note: Refer to the visible test case for better reference.
Sample Test Cases
titanicDat...
Submit
Explorer
Debugger
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
print(data.groupby('Pclass') ['Survived'].mean())
#2. Calculate the survival rate by embarked location (Embarked_S)
print(data.groupby ('Embarked_S') ['Survived'].mean())
#3. Calculate the survival rate by family size
print(data.groupby('FamilySize') ['Survived'].mean())
#4. Calculate the survival rate by being alone
print(data.groupby('IsAlone') ['Survived'].mean())
#5. Get the average fare by class
print(data.groupby('Pclass') ['Fare'].mean())
#6. Get the average age by class
print(data.groupby('Pclass') ['Age'].mean())
#7. Get the average age by survival status
Average time
0.062 s
62.00 ms
Maximum time
0.062 s
62.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
Actual output
Terminal
Test cases
Prev
Reset
Submit
Next

Course | Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6d/698c54860aba96214c8b7d6a/67e378a17020471d94dad0a6

CODETANTRA Home Learn Anywhere — 2025010900044@mitaoe.ac.in Support Logout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
2,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/02. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

```python
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'])

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print( mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print( median_fare)

# 6. Get the number of passengers by class
print( data['Pclass'].value_counts())

# 7. Get the number of passengers by gender
print( data['Sex'].value_counts())

# 8. Get the number of passengers by survival status
print( data['Survived'].value_counts())
```

Average time: 0.050 s (50.00 ms) | Maximum time: 0.050 s (50.00 ms) | 1 out of 1 shown test case(s) passed

Test case 1 — Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3 ... 491	3 ... 491
1 ... 216	1 ... 216
2 ... 184	2 ... 184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64

Terminal | Test cases

‹ Prev | Reset | Submit | Next ›

View recent browsing across windows and devices

Course | New Tab | GitHub Dashboard | View 1 · @pawangaikwad12's u | New Tab

FPS 126

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e2b9309025661df3aa574f

CODETANTRA Home Learn Anywhere

2025109004@mitaoe.ac.in Support Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

Submit

```
1  import pandas as pd
2  import numpy as np
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # 1. Display the first 5 rows of the dataset
8  print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 # 3. Get the shape of the dataset (number of rows and columns)
14 print(data.shape)
15
16 # 4. Get a summary of the dataset
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values and display the count
23 print(data.isnull().sum())
24
25 # 7. Fill missing values in the 'Age' column with the median age
26
27 data['Age'].fillna(data['Age'].median(), inplace=True)
28
29 # 8. Fill missing values in the 'Embarked' column with the most frequent value (mode)
30
31 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
32
33 # 9. Drop the 'Cabin' column due to many missing values
```

Average time: 0.180 s, 100.00 ms | Minimum time: 0.180 s, 100.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 — Debug

Expected output

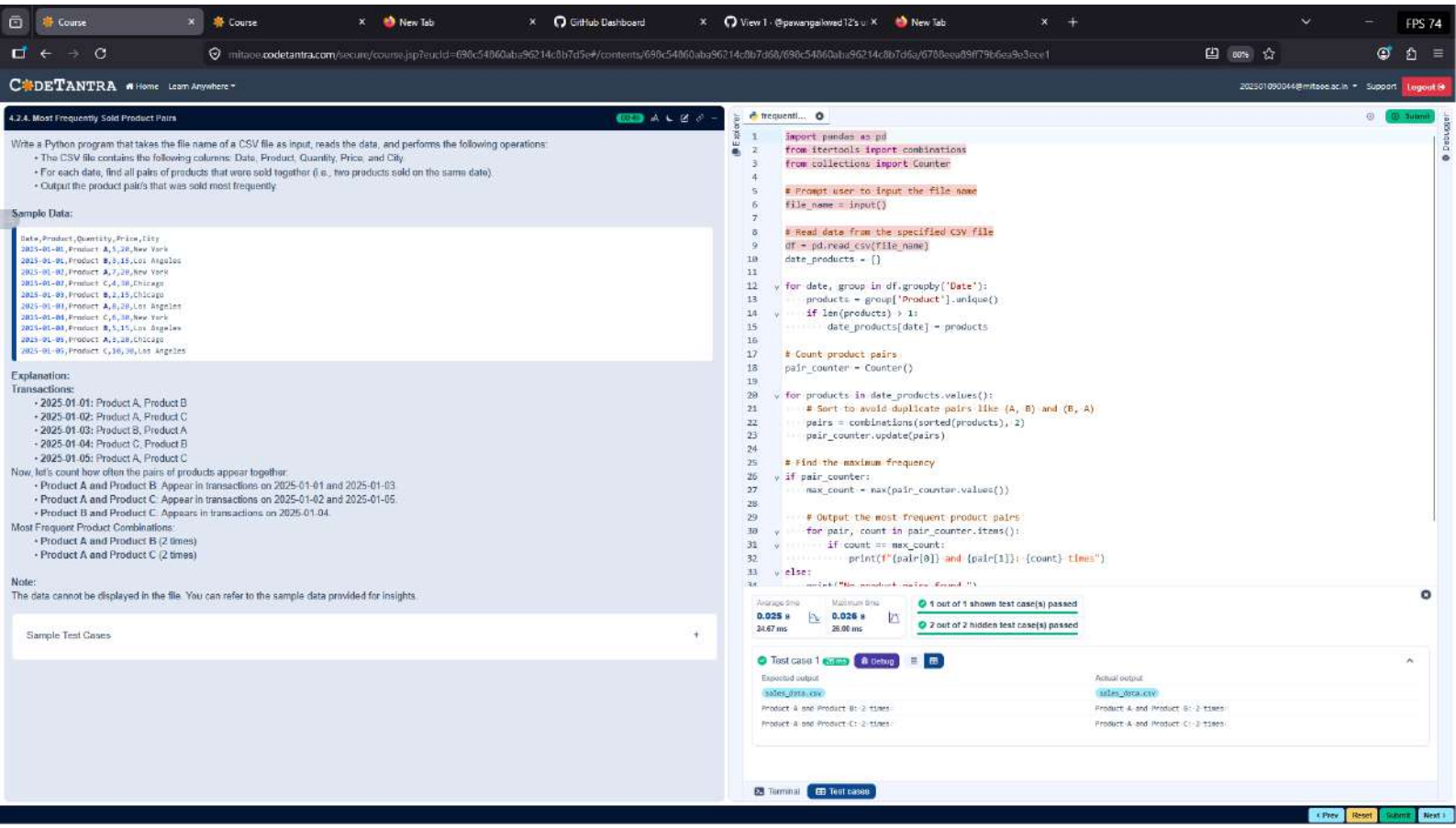
	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S

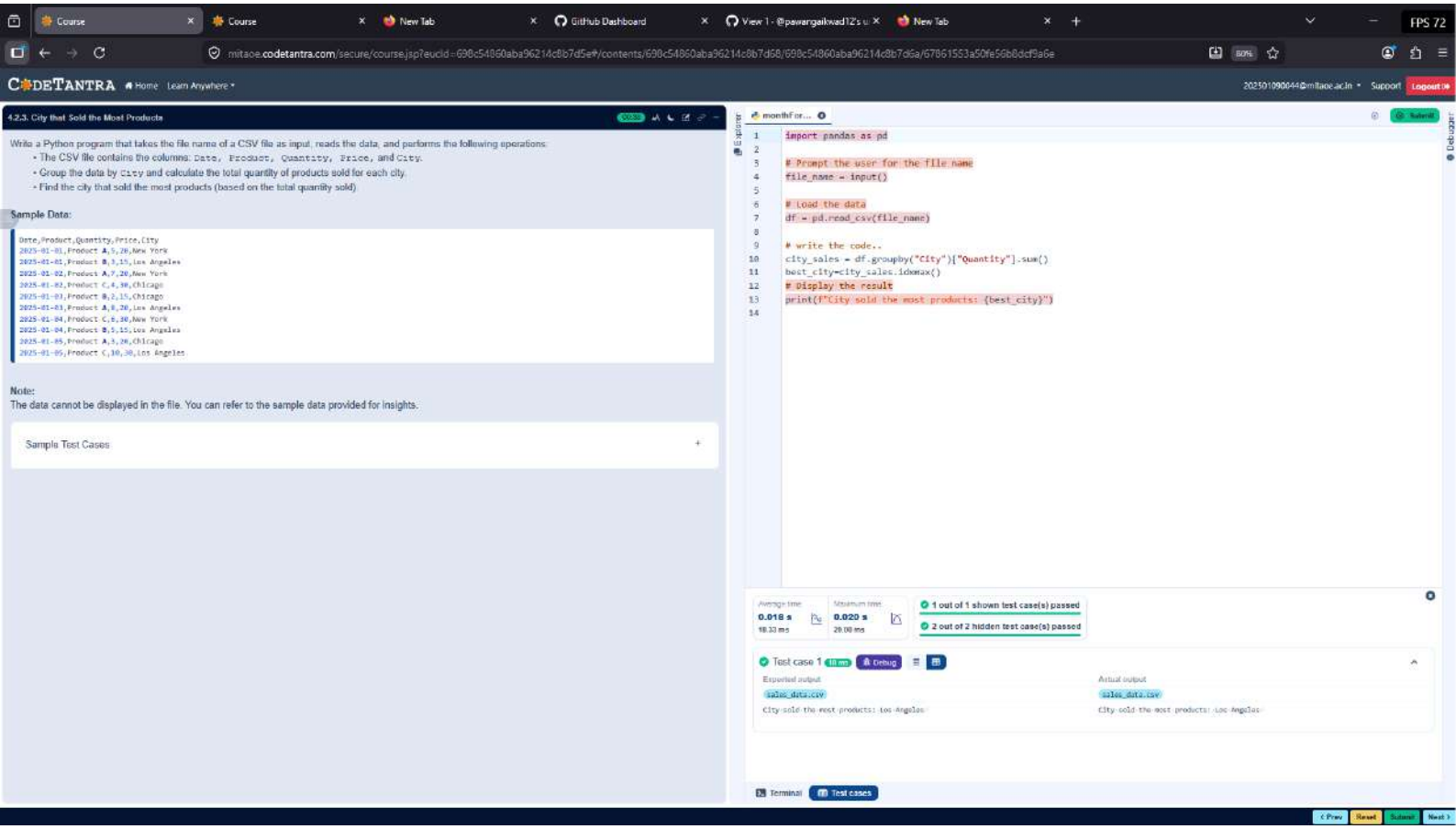
Actual output

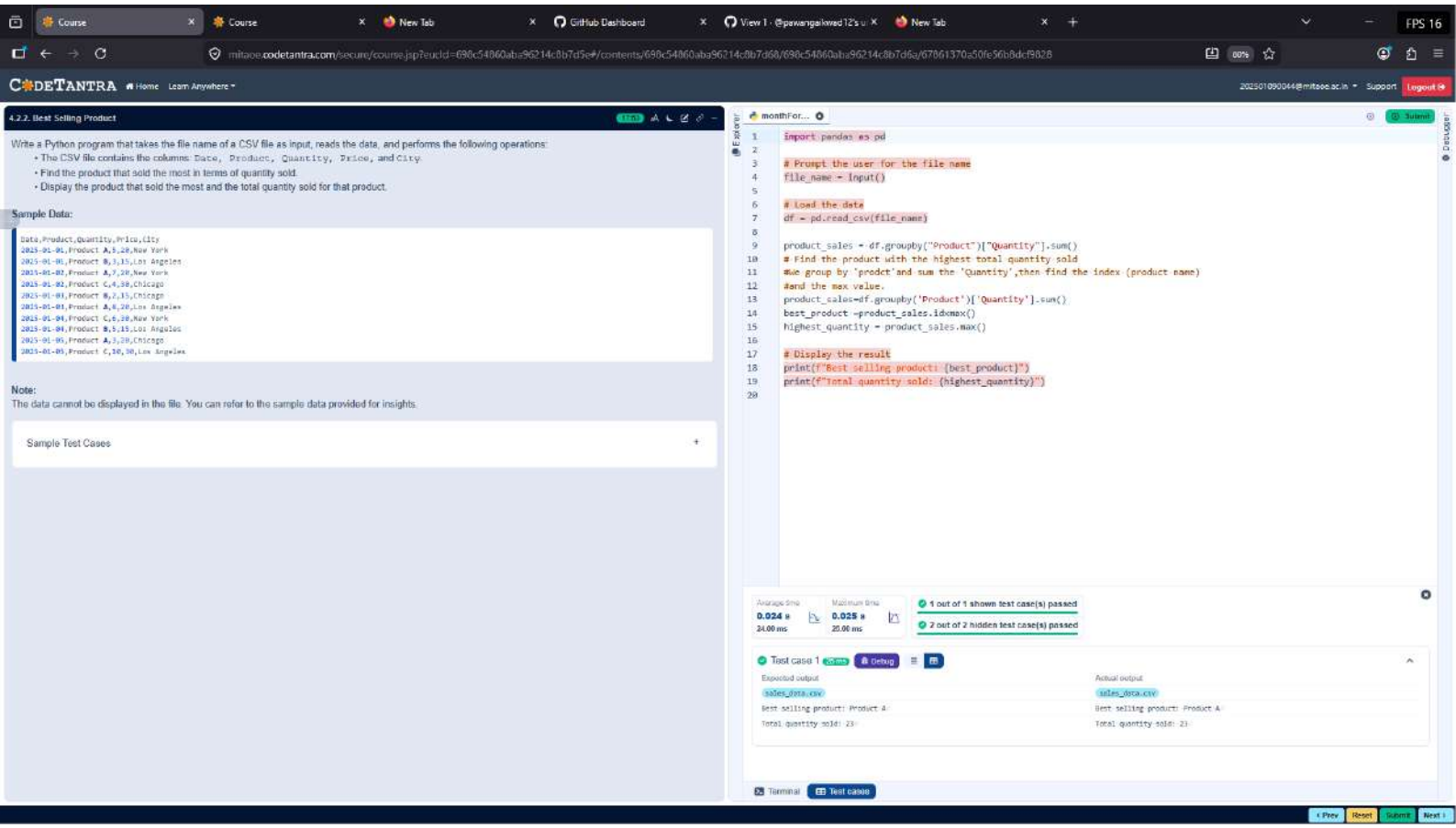
	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S

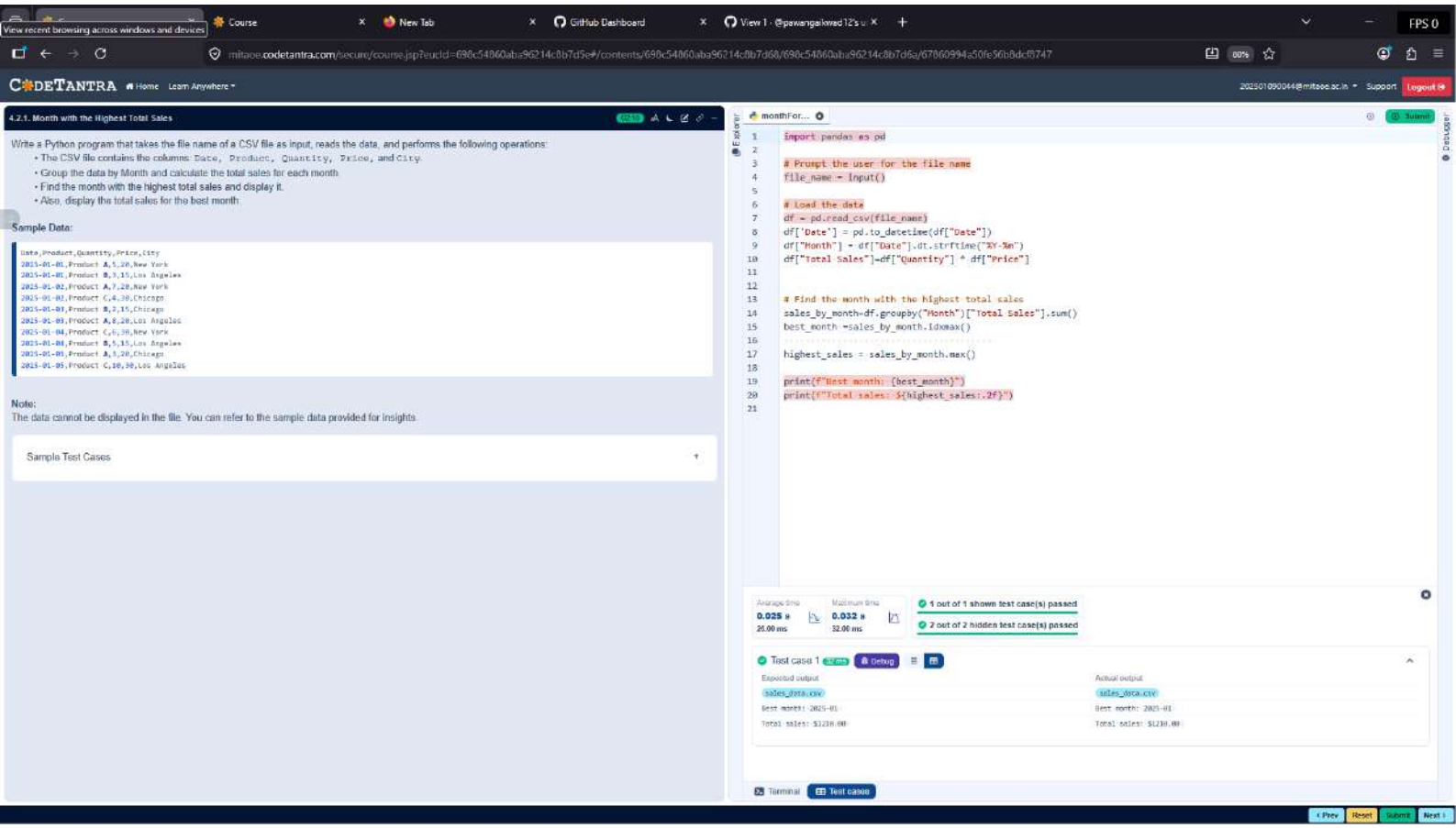
Terminal | Test cases

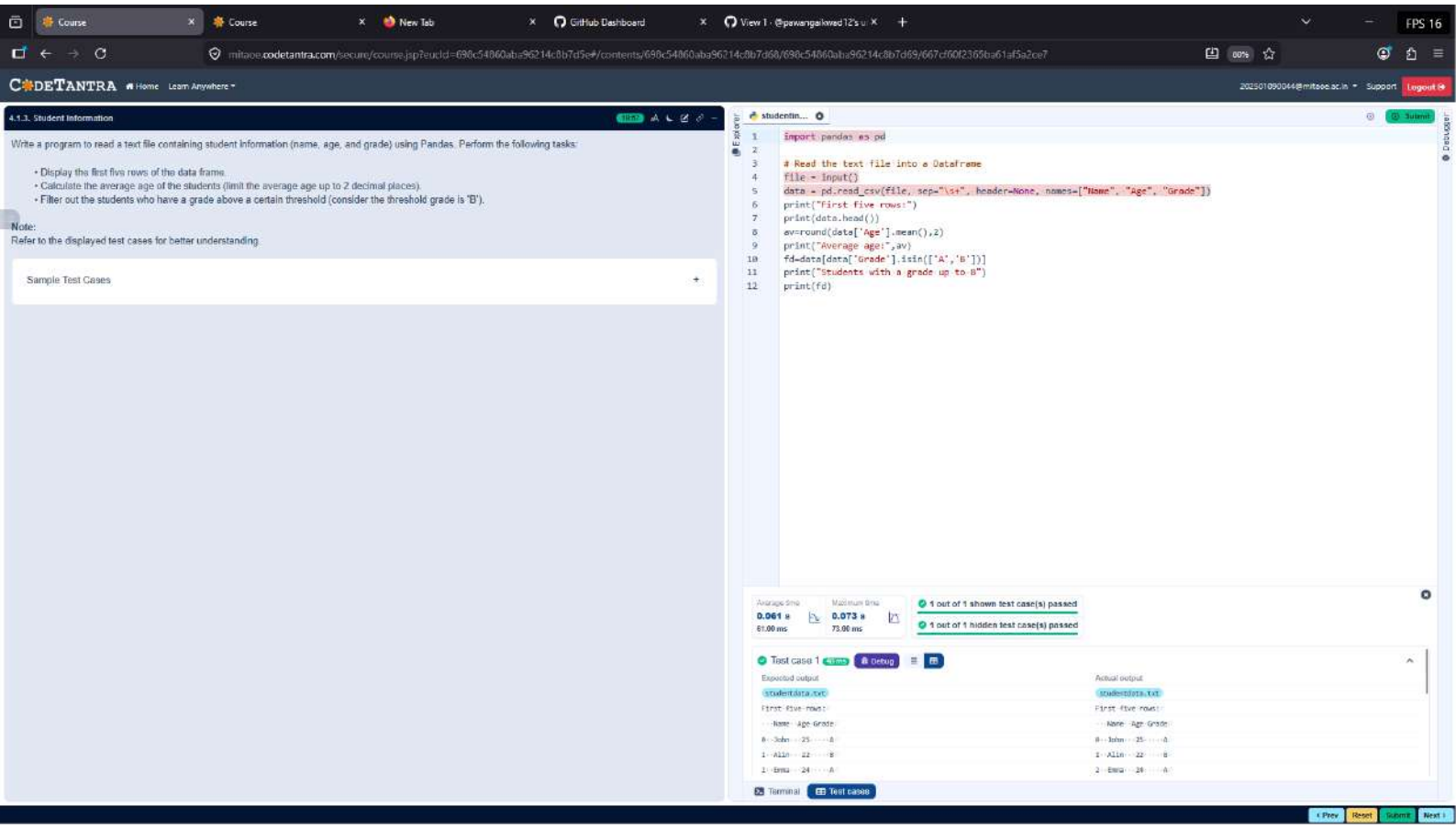
‹ Prev | Reset | Submit | Next ›

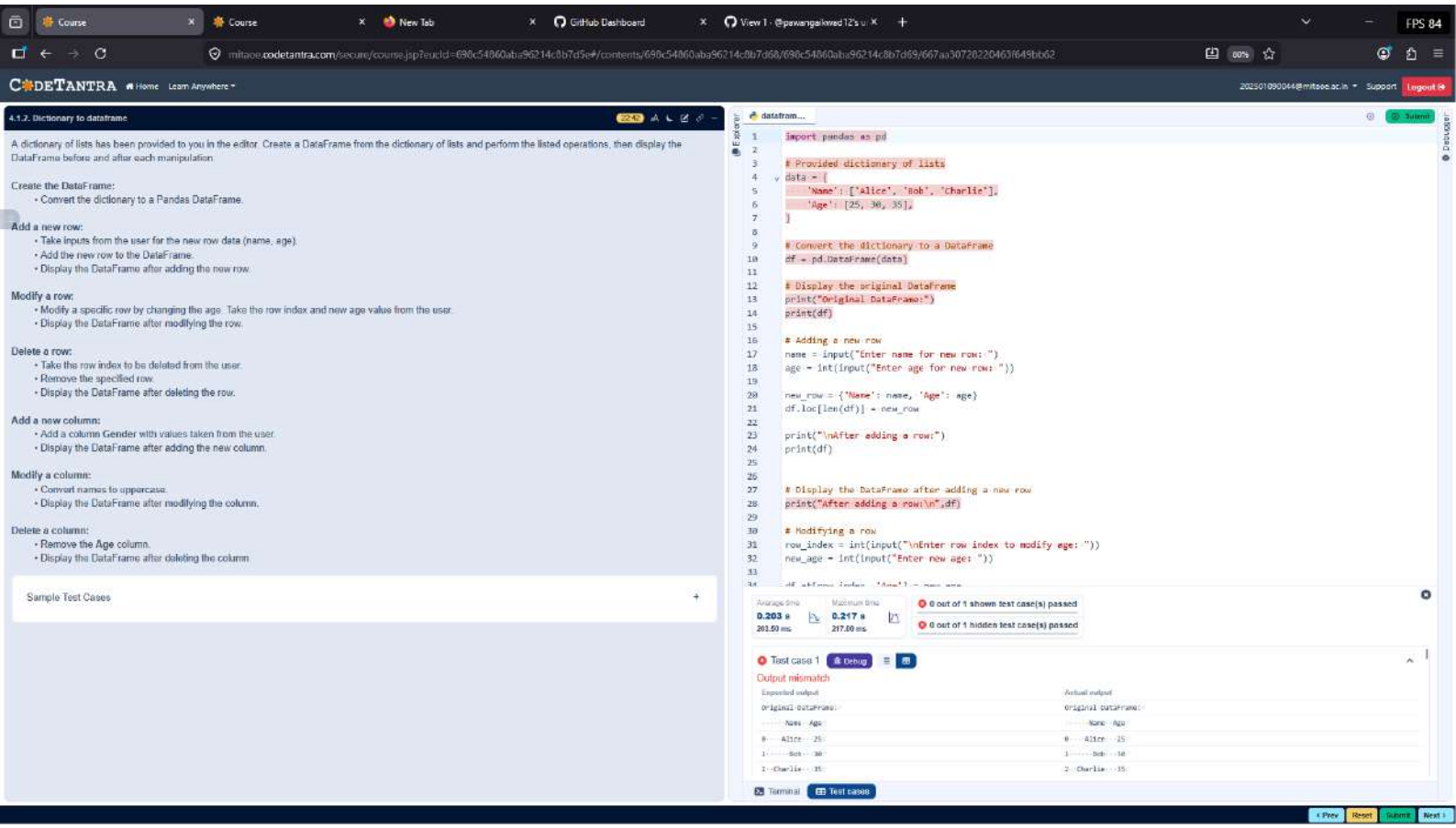

Course
Course
New Tab
GitHub Dashboard
New Tab
FPS 74
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d60/698c54860aba96214c8b7d6a/6788eea89ff79b6ea9e3ece1
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.4. Most Frequently Sold Product Pairs
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
• For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
• Output the product pair/s that was sold most frequently.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Explanation:
Transactions:
• 2025-01-01: Product A, Product B
• 2025-01-02: Product A, Product C
• 2025-01-03: Product B, Product A
• 2025-01-04: Product C, Product B
• 2025-01-05: Product A, Product C
Now, let's count how often the pairs of products appear together:
• Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
• Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
• Product B and Product C: Appears in transactions on 2025-01-04.
Most Frequent Product Combinations:
• Product A and Product B (2 times)
• Product A and Product C (2 times)
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
frequenti...
Submit
Explorer
Debugger
import pandas as pd
from itertools import combinations
from collections import Counter
# Prompt user to input the file name
file_name = input()
# Read data from the specified CSV file
df = pd.read_csv(file_name)
date_products = {}
for date, group in df.groupby('Date'):
products = group['Product'].unique()
if len(products) > 1:
date_products[date] = products
# Count product pairs
pair_counter = Counter()
for products in date_products.values():
# Sort to avoid duplicate pairs like (A, B) and (B, A)
pairs = combinations(sorted(products), 2)
pair_counter.update(pairs)
# Find the maximum frequency
if pair_counter:
max_count = max(pair_counter.values())
# Output the most frequent product pairs
for pair, count in pair_counter.items():
if count == max_count:
print(f"{pair[0]} and {pair[1]}: {count} times")
else:
print("No product pairs found.")
Average time
0.025 s
24.67 ms
Maximum time
0.026 s
26.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
sales_data.csv
Product A and Product B: 2 times
Product A and Product C: 2 times
Actual output
sales_data.csv
Product A and Product B: 2 times
Product A and Product C: 2 times
Terminal
Test cases
Prev
Reset
Submit
Next

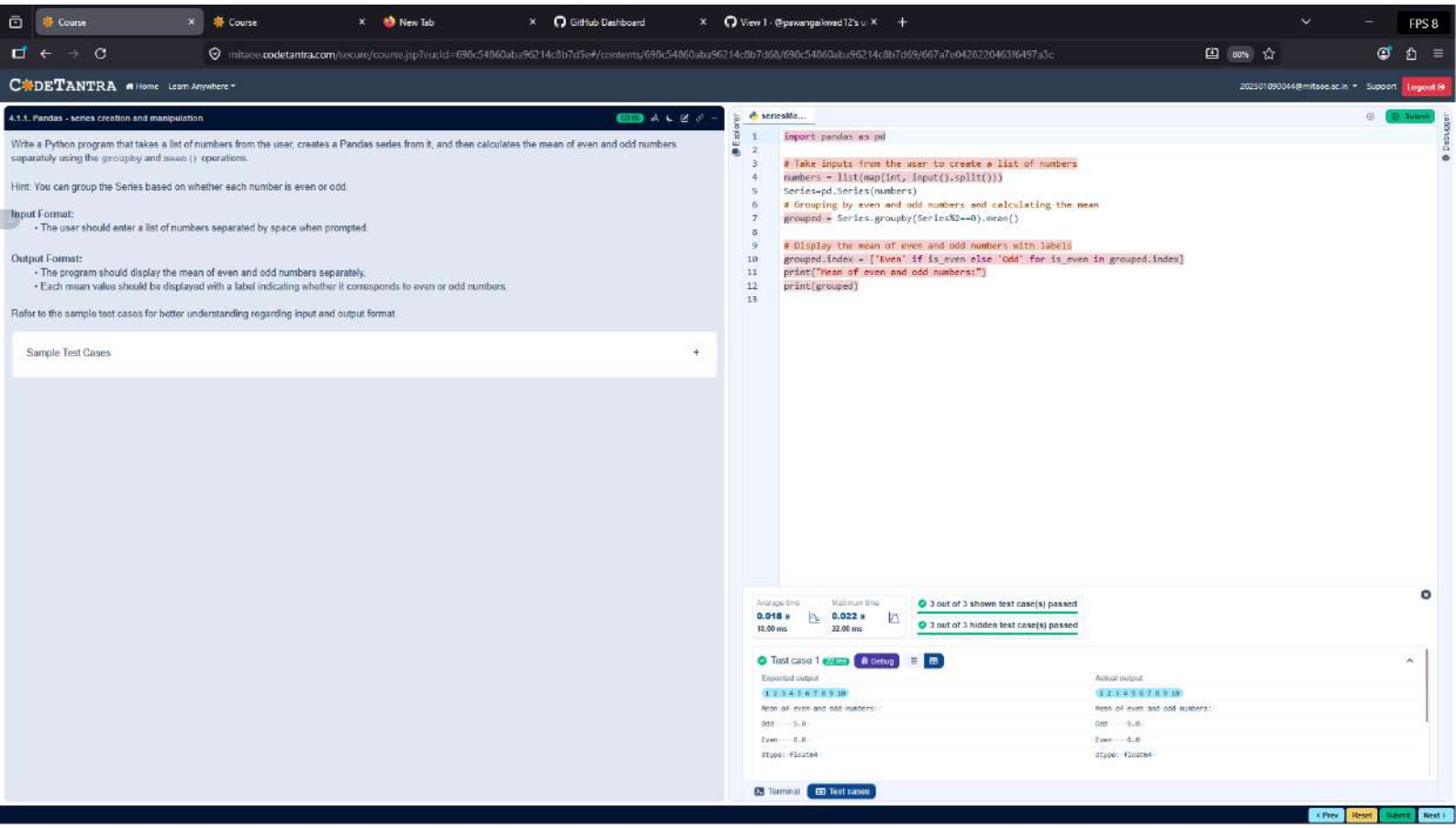

Course
Course
New Tab
Github Dashboard
View 1 - @pawargaikwad1Z's u
New Tab
FPS 72
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d58/698c54860aba96214c8b7d6a/67861553a50fe56b8dcf9a6e
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.3. City that Sold the Most Products
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Group the data by City and calculate the total quantity of products sold for each city.
• Find the city that sold the most products (based on the total quantity sold)
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
monthFor...
Submit
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# write the code..
city_sales = df.groupby("City")["Quantity"].sum()
best_city=city_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
Average time
0.018 s
18.33 ms
Minimum time
0.020 s
20.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
sales_data.csv
City sold the most products: Los Angeles
Actual output
sales_data.csv
City sold the most products: Los Angeles
Terminal
Test cases
Prev
Reset
Submit
Next

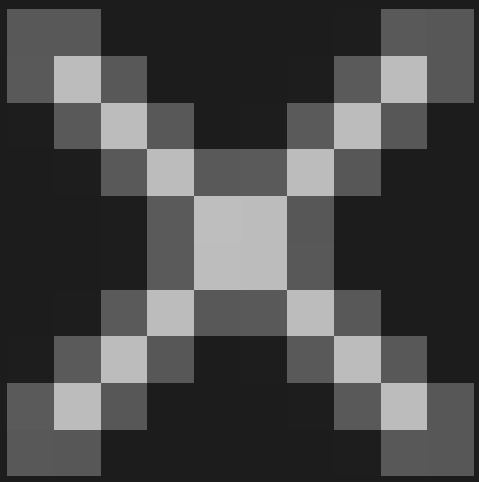

Course
Course
New Tab
GitHub Dashboard
New Tab
FPS 16
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d60/698c54860aba96214c8b7d6a/67861370a50fe56b8dcf8028
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.2. Best Selling Product
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Find the product that sold the most in terms of quantity sold.
• Display the product that sold the most and the total quantity sold for that product.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,6,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
monthFor...
Submit
Explorer
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
#we group by 'prodct'and sum the 'Quantity',then find the index (product name)
#and the max value.
product_sales=df.groupby('Product')['Quantity'].sum()
best_product =product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
Average time
0.024 s
24.00 ms
Maximum time
0.025 s
25.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Actual output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Terminal
Test cases
Prev
Reset
Submit
Next

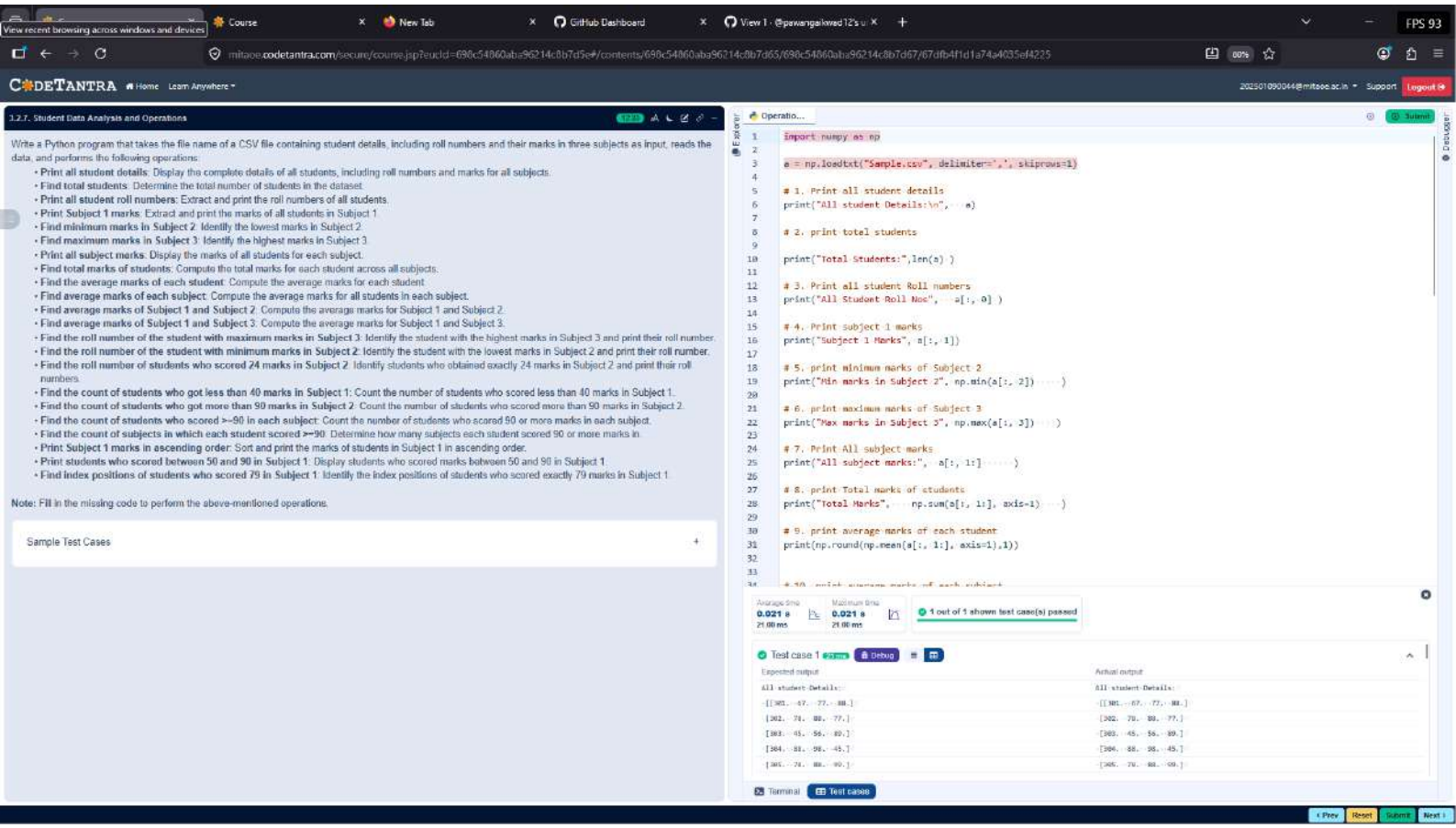

View recent browsing across windows and devices
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 0
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67860994a50fe56b8dcf8747
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.2.1. Month with the Highest Total Sales
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Group the data by Month and calculate the total sales for each month.
• Find the month with the highest total sales and display it.
• Also, display the total sales for the best month.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,6,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
monthFor...
Submit
Explorer
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df["Date"])
df["Month"] = df["Date"].dt.strftime("%Y-%m")
df["Total Sales"]=df["Quantity"] * df["Price"]
# Find the month with the highest total sales
sales_by_month=df.groupby("Month")["Total Sales"].sum()
best_month =sales_by_month.idxmax()
highest_sales = sales_by_month.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
Average time
0.025 s
20.00 ms
Maximum time
0.032 s
32.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
sales_data.csv
Best month: 2025-01
Total sales: $1230.00
Actual output
sales_data.csv
Best month: 2025-01
Total sales: $1230.00
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

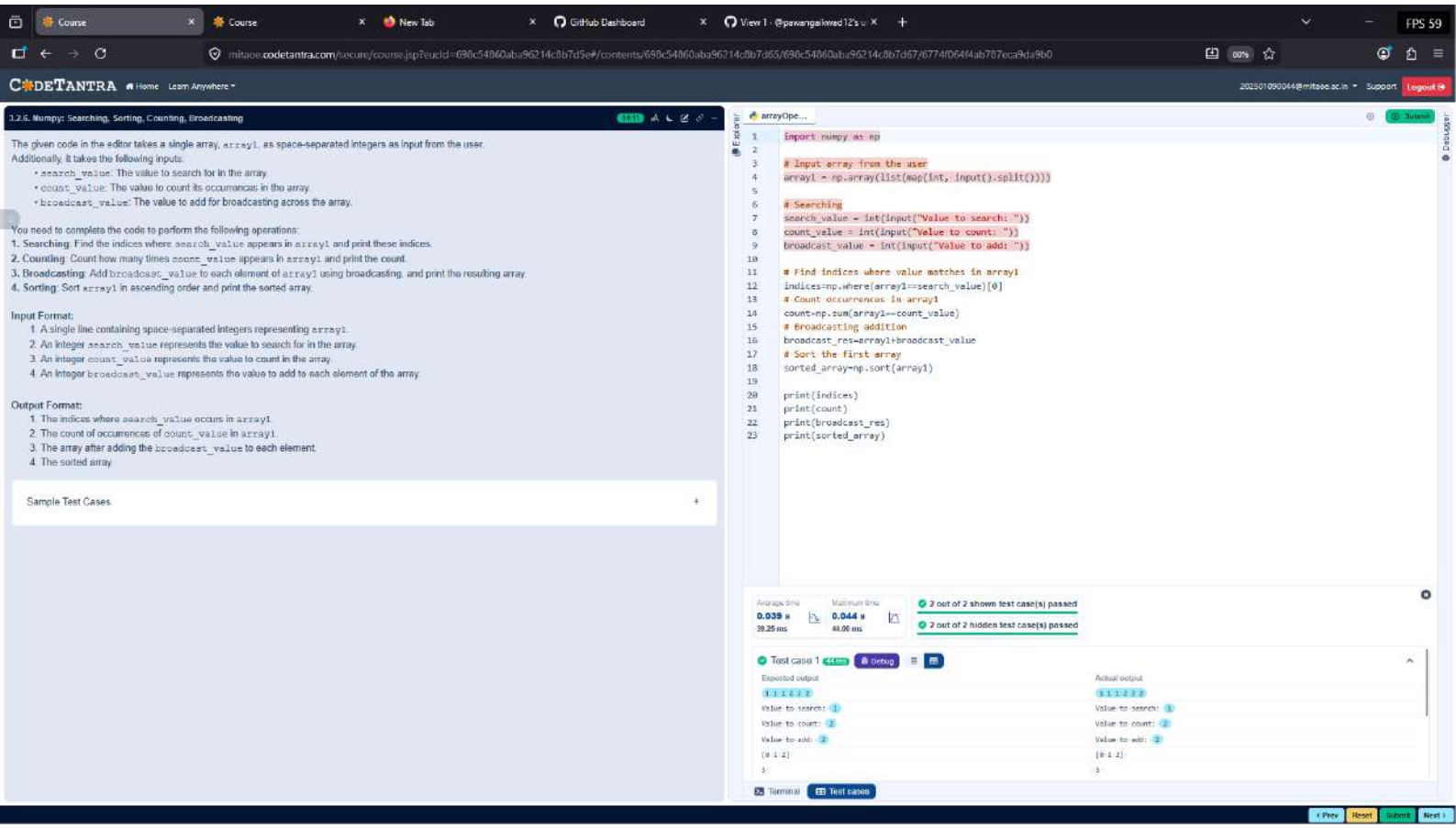

Course
Course
New Tab
GitHub Dashboard
FPS 16
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d68/667cf60f2365ba61af5a2ce7
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.1.3. Student Information
Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:
• Display the first five rows of the data frame.
• Calculate the average age of the students (limit the average age up to 2 decimal places).
• Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').
Note:
Refer to the displayed test cases for better understanding.
Sample Test Cases
import pandas as pd
# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
print("First five rows:")
print(data.head())
av=round(data['Age'].mean(),2)
print("Average age:",av)
fd=data[data['Grade'].isin(['A','B'])]
print("Students with a grade up to B")
print(fd)
0.061 s
61.00 ms
0.073 s
73.00 ms
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
studentdata.txt
First five rows:
Name Age Grade
0 John 25 A
1 Alin 22 B
2 Emma 24 A
Terminal
Test cases
Prev
Reset
Submit
Next

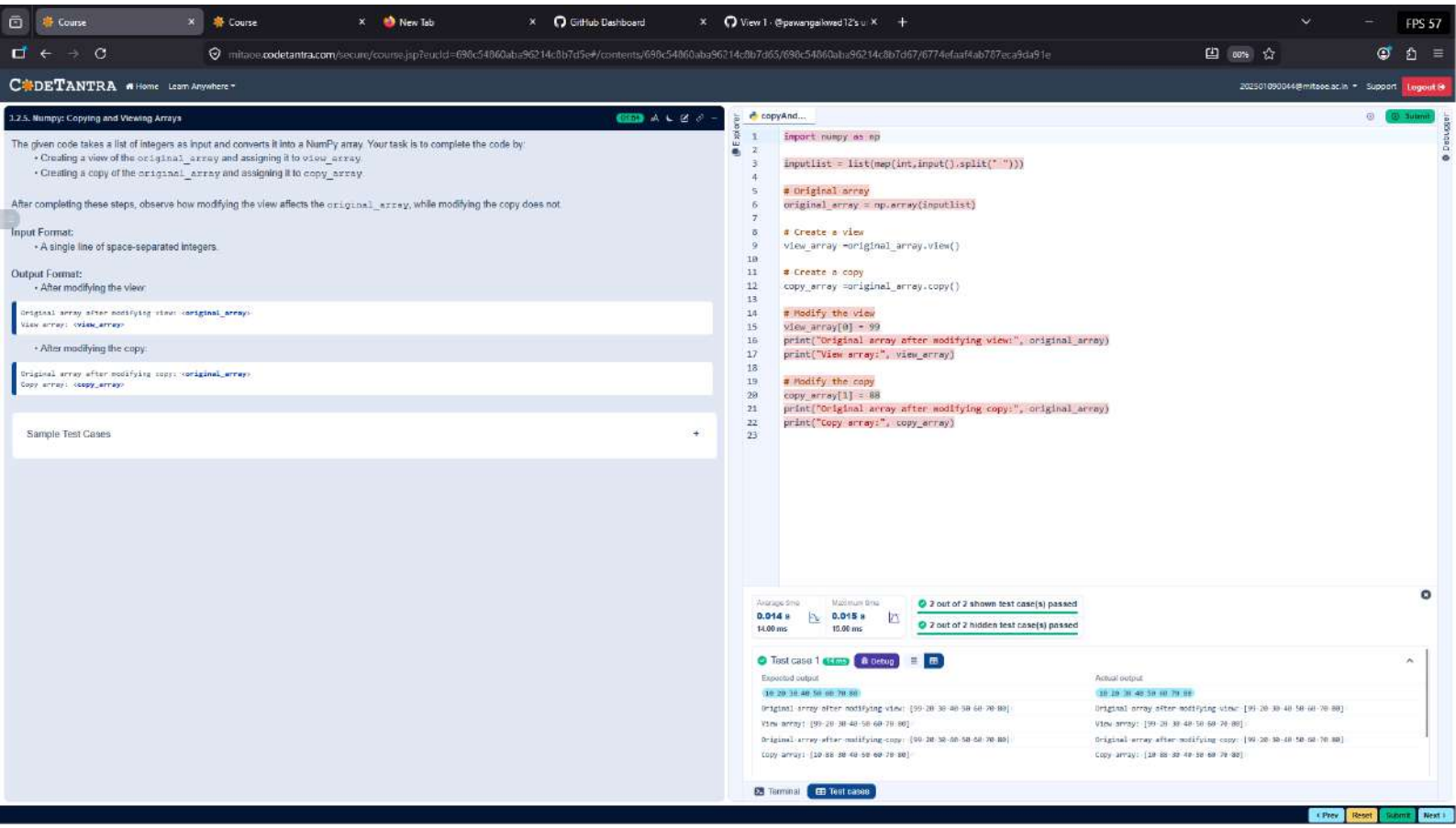

Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 84
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667aa307262204636f649bb62
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.1.2. Dictionary to dataframe
A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.
Create the DataFrame:
• Convert the dictionary to a Pandas DataFrame.
Add a new row:
• Take inputs from the user for the new row data (name, age).
• Add the new row to the DataFrame.
• Display the DataFrame after adding the new row.
Modify a row:
• Modify a specific row by changing the age. Take the row index and new age value from the user.
• Display the DataFrame after modifying the row.
Delete a row:
• Take the row index to be deleted from the user.
• Remove the specified row.
• Display the DataFrame after deleting the row.
Add a new column:
• Add a column Gender with values taken from the user.
• Display the DataFrame after adding the new column.
Modify a column:
• Convert names to uppercase.
• Display the DataFrame after modifying the column.
Delete a column:
• Remove the Age column.
• Display the DataFrame after deleting the column.
Sample Test Cases
dataframe...
Submit
Explorer
Debugger
import pandas as pd
# Provided dictionary of lists
data = {
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35],
}
# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
# Display the original DataFrame
print("Original DataFrame:")
print(df)
# Adding a new row
name = input("Enter name for new row: ")
age = int(input("Enter age for new row: "))
new_row = {'Name': name, 'Age': age}
df.loc[len(df)] = new_row
print("\nAfter adding a row:")
print(df)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
# Modifying a row
row_index = int(input("\nEnter row index to modify age: "))
new_age = int(input("Enter new age: "))
Average time
0.203 s
203.50 ms
Maximum time
0.217 s
217.00 ms
0 out of 1 shown test case(s) passed
0 out of 1 hidden test case(s) passed
Test case 1
Debug
Output mismatch
Expected output
Original DataFrame:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
Actual output
Original DataFrame:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

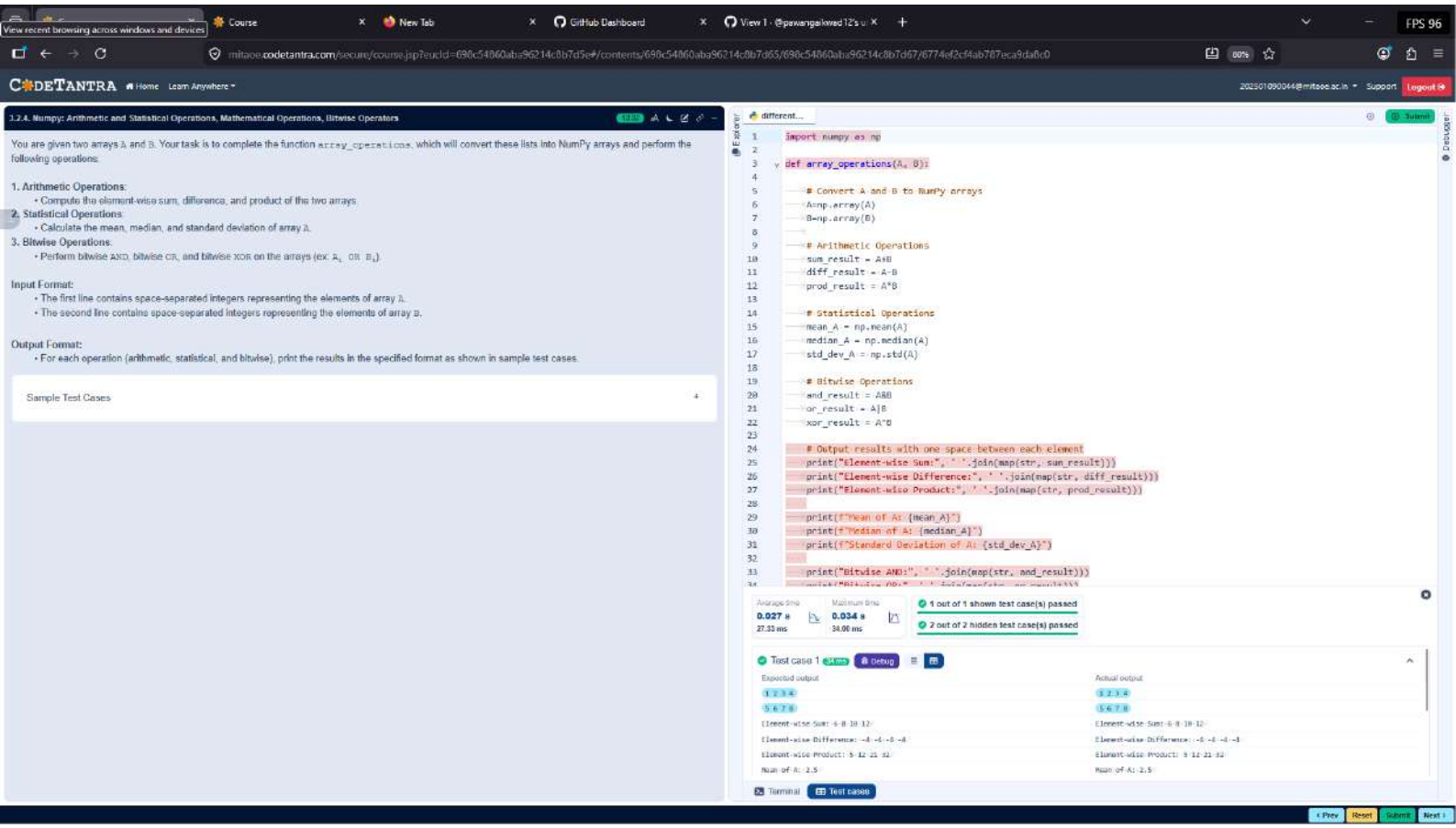

Course
Course
New Tab
GitHub Dashboard
FPS 8
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667a7e0420220463f6487a3c
CODETANTRA
Home
Learn Anywhere
Support
Logout
4.1.1. Pandas - series creation and manipulation
Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.
Hint: You can group the Series based on whether each number is even or odd.
Input Format:
• The user should enter a list of numbers separated by space when prompted.
Output Format:
• The program should display the mean of even and odd numbers separately.
• Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.
Refer to the sample test cases for better understanding regarding input and output format
Sample Test Cases
Explorer
seriesMe...
Submit
Debugger
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5 Series=pd.Series(numbers)
6 # Grouping by even and odd numbers and calculating the mean
7 grouped = Series.groupby(Series%2==0).mean()
8
9 # Display the mean of even and odd numbers with labels
10 grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
11 print("Mean of even and odd numbers:")
12 print(grouped)
13
Average time
0.018 s
10.00 ms
Maximum time
0.022 s
22.00 ms
3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
Debug
Expected output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64
Actual output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64
Terminal
Test cases
Prev
Reset
Submit
Next

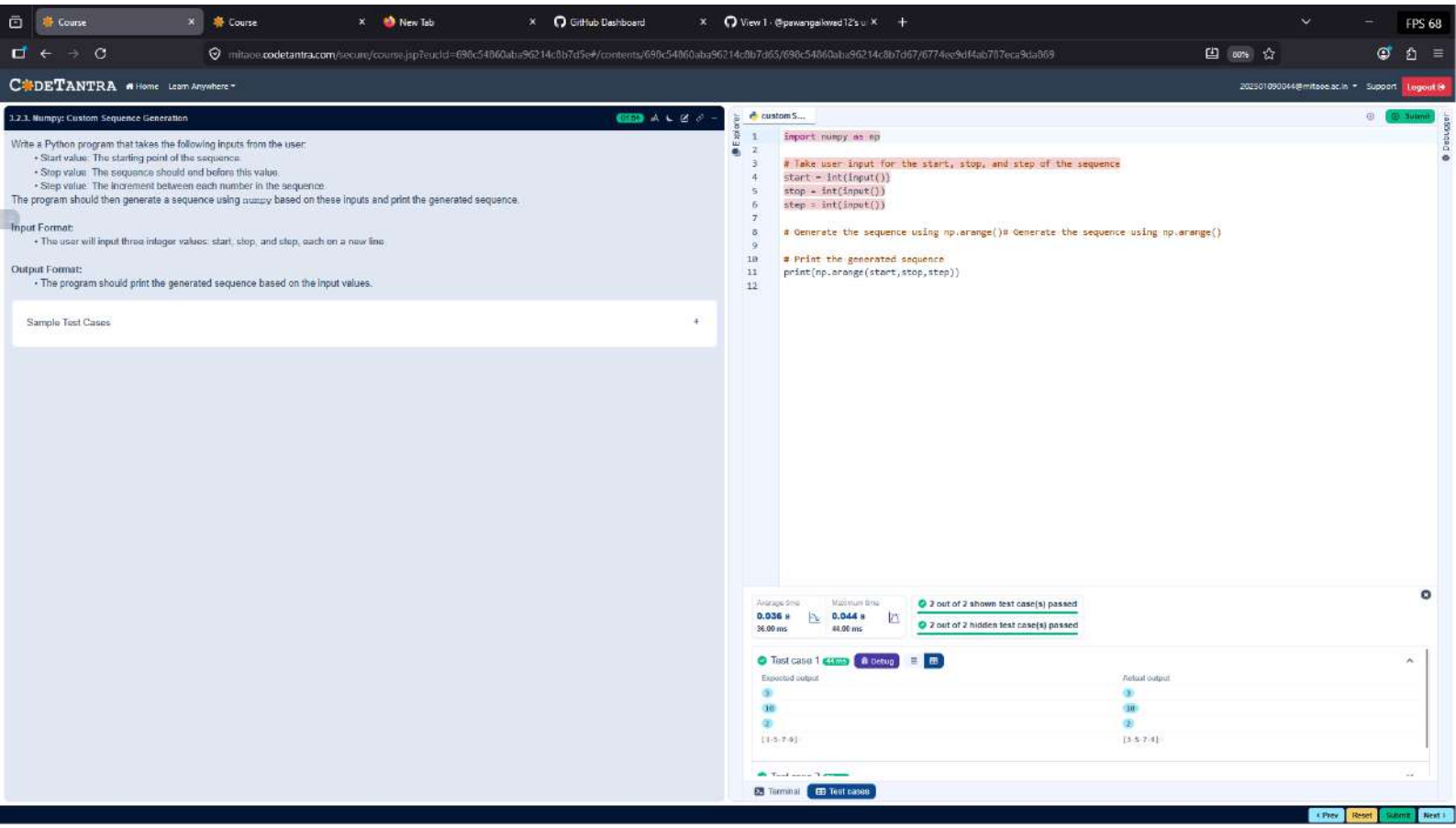


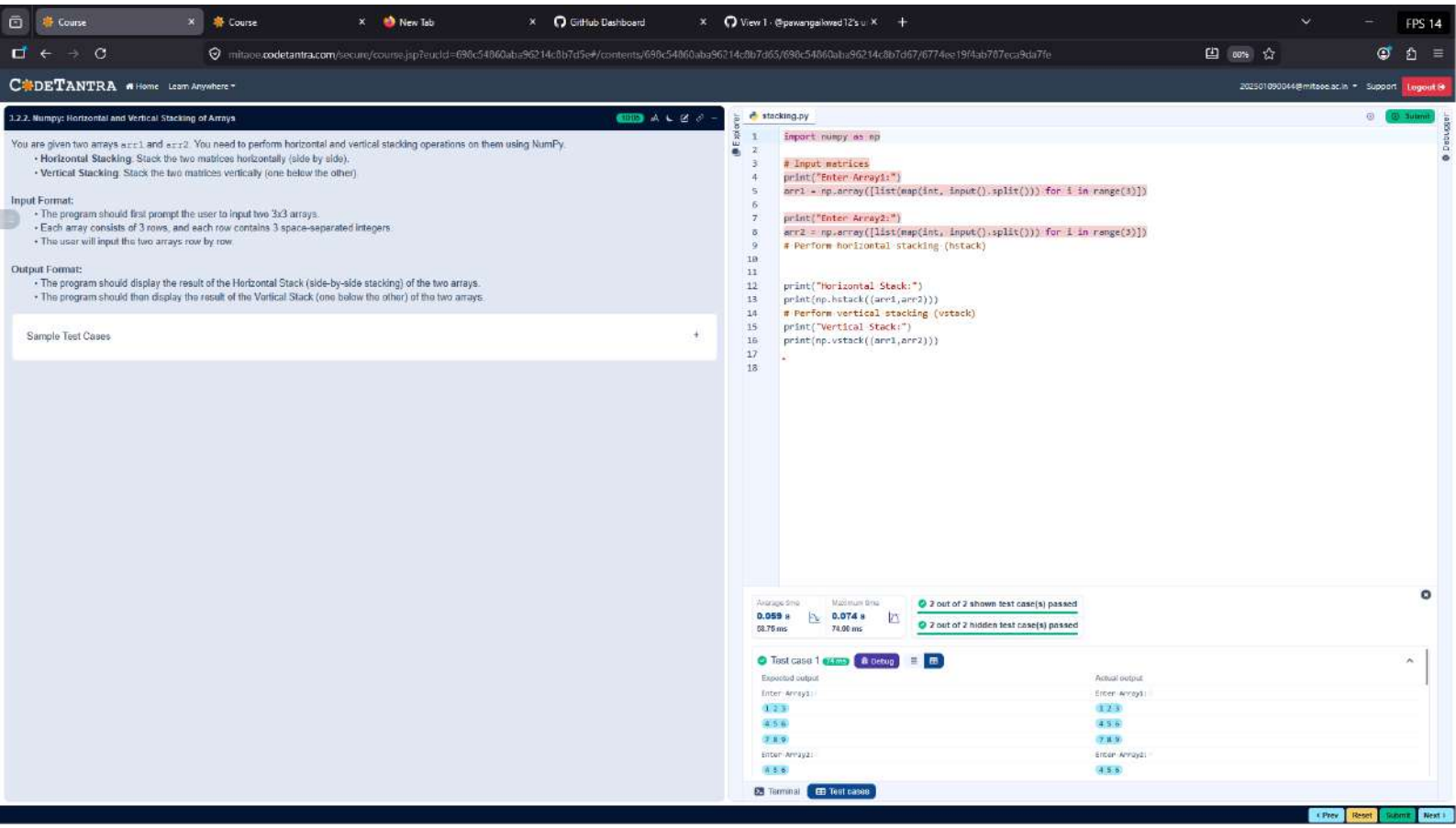

View recent browsing across windows and devices
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 93
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/67dfb4f1d1a74a4035ef4225
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.2.7. Student Data Analysis and Operations
Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:
• Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
• Find total students: Determine the total number of students in the dataset.
• Print all student roll numbers: Extract and print the roll numbers of all students.
• Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
• Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
• Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
• Print all subject marks: Display the marks of all students for each subject.
• Find total marks of students: Compute the total marks for each student across all subjects.
• Find the average marks of each student: Compute the average marks for each student.
• Find average marks of each subject: Compute the average marks for all students in each subject.
• Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
• Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
• Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
• Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
• Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
• Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
• Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
• Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
• Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.
• Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
• Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
• Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.
Note: Fill in the missing code to perform the above-mentioned operations.
Sample Test Cases
Operatio...
Submit
Explorer
Debugger
import numpy as np
a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
# 1. Print all student details
print("All student Details:\n", a)
# 2. print total students
print("Total Students:",len(a))
# 3. Print all student Roll numbers
print("All Student Roll Nos", a[:, 0])
# 4. Print subject 1 marks
print("Subject 1 Marks", a[:, 1])
# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(a[:, 2]))
# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(a[:, 3]))
# 7. Print All subject marks
print("All subject marks:", a[:, 1:])
# 8. print Total marks of students
print("Total Marks", np.sum(a[:, 1:], axis=1))
# 9. print average marks of each student
print(np.round(np.mean(a[:, 1:], axis=1),1))
Average time
0.021 s
21.00 ms
Maximum time
0.021 s
21.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
All student Details:
[[301. 67. 77. 88.]
[302. 78. 88. 77.]
[303. 45. 56. 89.]
[304. 88. 98. 45.]
[305. 78. 88. 99.]]
Actual output
All student Details:
[[301. 67. 77. 88.]
[302. 78. 88. 77.]
[303. 45. 56. 89.]
[304. 88. 98. 45.]
[305. 78. 88. 99.]]
Terminal
Test cases
Prev
Reset
Submit
Next

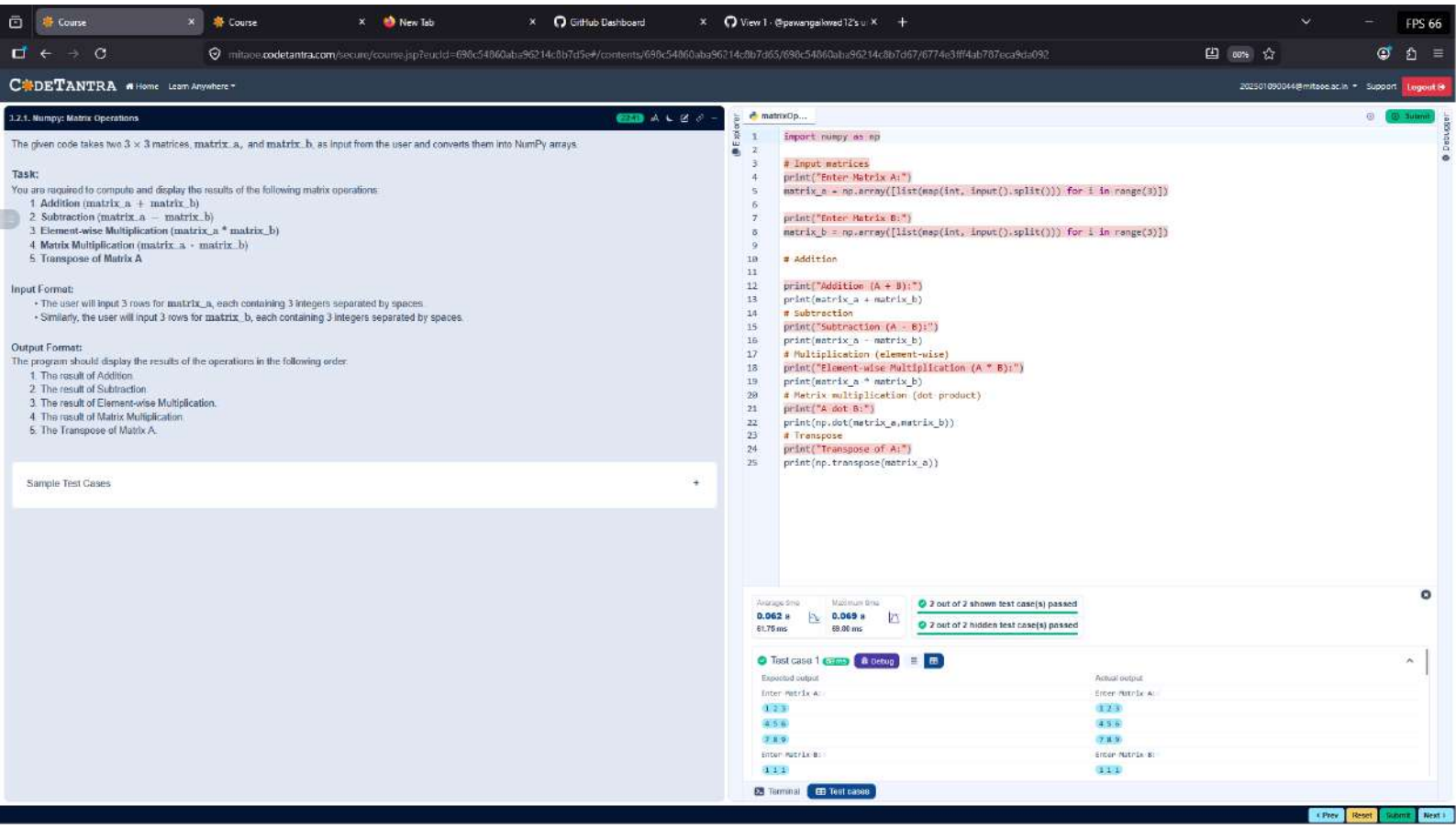

Course
Course
New Tab
GitHub Dashboard
FPS 59
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774f064f4ab787eca9da9b0
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.2.6. Numpy: Searching, Sorting, Counting, Broadcasting
The given code in the editor takes a single array, array1, as space-separated integers as input from the user.
Additionally, it takes the following inputs:
search_value: The value to search for in the array.
count_value: The value to count its occurrences in the array.
broadcast_value: The value to add for broadcasting across the array.
You need to complete the code to perform the following operations:
1. Searching: Find the indices where search_value appears in array1 and print these indices.
2. Counting: Count how many times count_value appears in array1 and print the count.
3. Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. Sorting: Sort array1 in ascending order and print the sorted array.
Input Format:
1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.
Output Format:
1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.
Sample Test Cases
arrayOpe...
Submit
import numpy as np
# Input array from the user
array1 = np.array(list(map(int, input().split())))
# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))
# Find indices where value matches in array1
indices=np.where(array1==search_value)[0]
# Count occurrences in array1
count=np.sum(array1==count_value)
# Broadcasting addition
broadcast_res=array1+broadcast_value
# Sort the first array
sorted_array=np.sort(array1)
print(indices)
print(count)
print(broadcast_res)
print(sorted_array)
Average time
0.039 s
39.25 ms
Maximum time
0.044 s
44.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
44 ms
Debug
Expected output
1 1 1 2 2
Value to search: 1
Value to count: 2
Value to add: 2
[0 1 2]
3
Actual output
1 1 1 2 2
Value to search: 1
Value to count: 2
Value to add: 2
[0 1 2]
3
Terminal
Test cases
Prev
Reset
Submit
Next

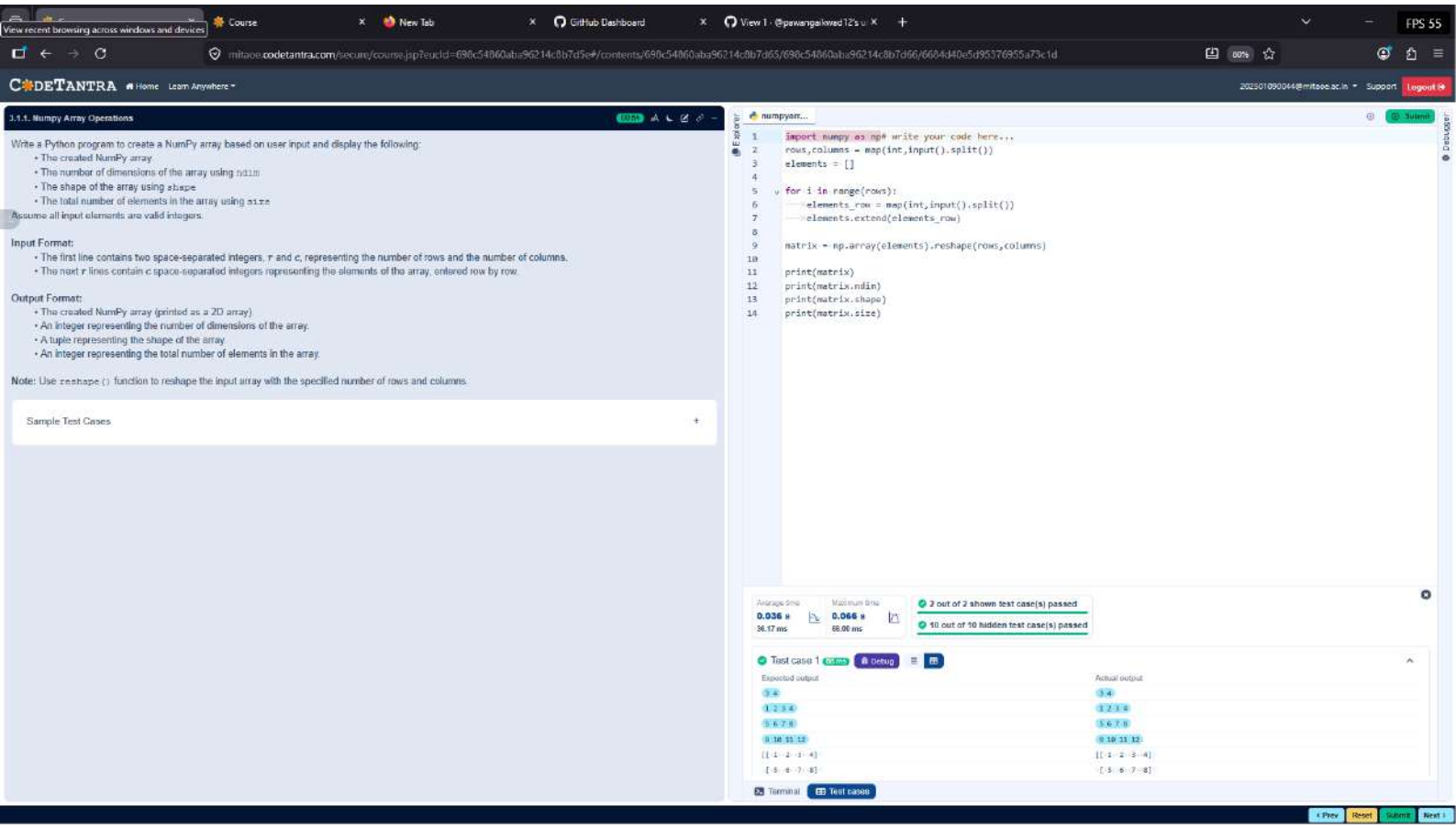

Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 57
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774efaaf4ab787eca9da91e
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.2.5. Numpy: Copying and Viewing Arrays
The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:
• Creating a view of the original_array and assigning it to view_array.
• Creating a copy of the original_array and assigning it to copy_array.
After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.
Input Format:
• A single line of space-separated integers.
Output Format:
• After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
• After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>
Sample Test Cases
copyAnd...
Submit
Explorer
Debugger
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array =original_array.view()
10
11 # Create a copy
12 copy_array =original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21 print("Original array after modifying copy:", original_array)
22 print("Copy array:", copy_array)
23
Average time
0.014 s
14.00 ms
Maximum time
0.015 s
15.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
14 ms
Debug
Expected output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Actual output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

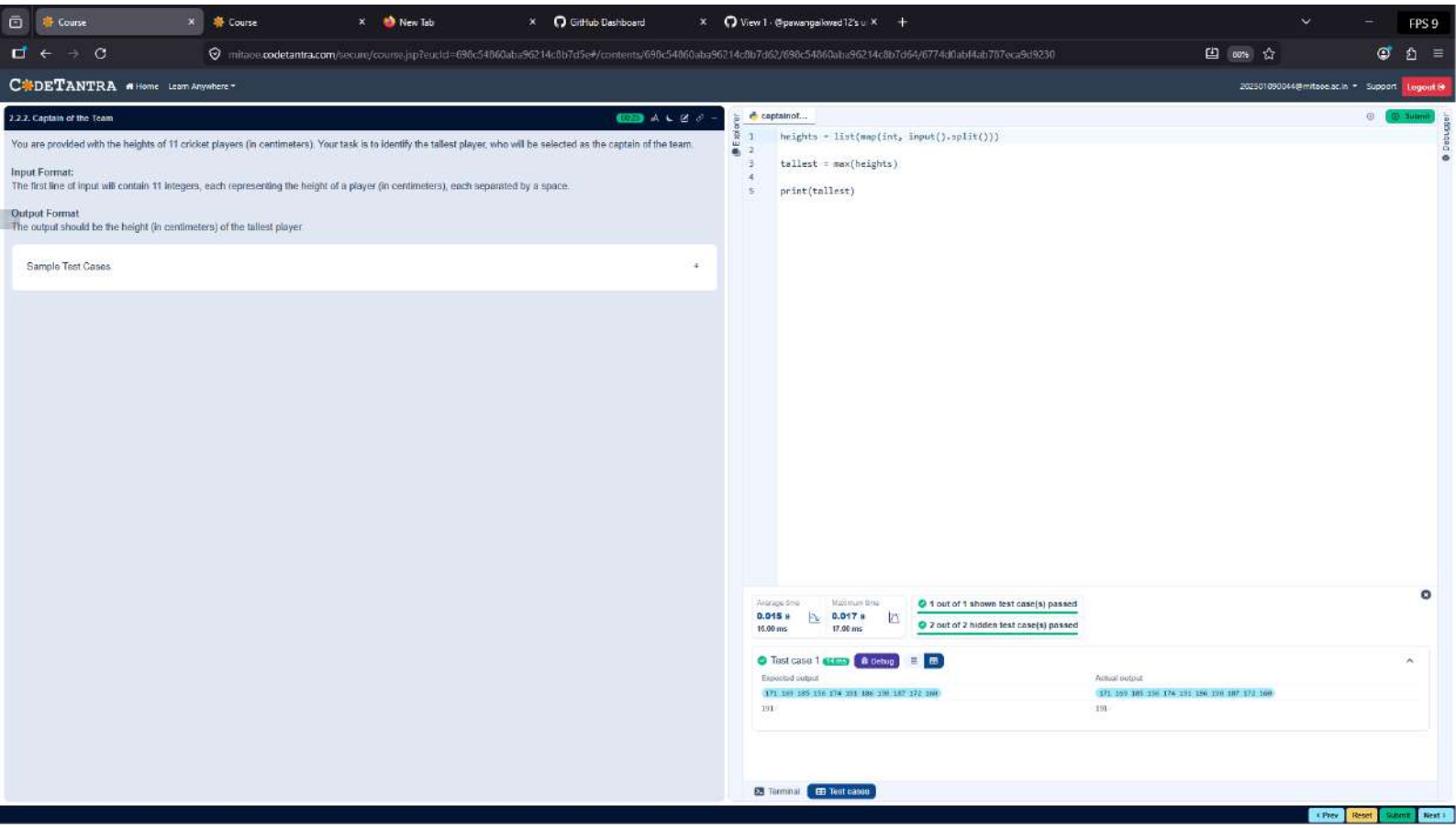

Course
New Tab
GitHub Dashboard
FPS 96
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ef2cf4ab787eca9da8c0
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:
1. Arithmetic Operations:
• Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
• Calculate the mean, median, and standard deviation of array A.
3. Bitwise Operations:
• Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A1 OR B1).
Input Format:
• The first line contains space-separated integers representing the elements of array A.
• The second line contains space-separated integers representing the elements of array B.
Output Format:
• For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.
Sample Test Cases
different...
Submit
Explorer
Debugger
import numpy as np
def array_operations(A, B):
# Convert A and B to NumPy arrays
A=np.array(A)
B=np.array(B)
# Arithmetic Operations
sum_result = A+B
diff_result = A-B
prod_result = A*B
# Statistical Operations
mean_A = np.mean(A)
median_A = np.median(A)
std_dev_A = np.std(A)
# Bitwise Operations
and_result = A&B
or_result = A|B
xor_result = A^B
# Output results with one space between each element
print("Element-wise Sum:", ' '.join(map(str, sum_result)))
print("Element-wise Difference:", ' '.join(map(str, diff_result)))
print("Element-wise Product:", ' '.join(map(str, prod_result)))
print(f"Mean of A: {mean_A}")
print(f"Median of A: {median_A}")
print(f"Standard Deviation of A: {std_dev_A}")
print("Bitwise AND:", ' '.join(map(str, and_result)))
Average time
0.027 s
27.33 ms
Maximum time
0.034 s
34.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
34 ms
Debug
Expected output
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Actual output
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

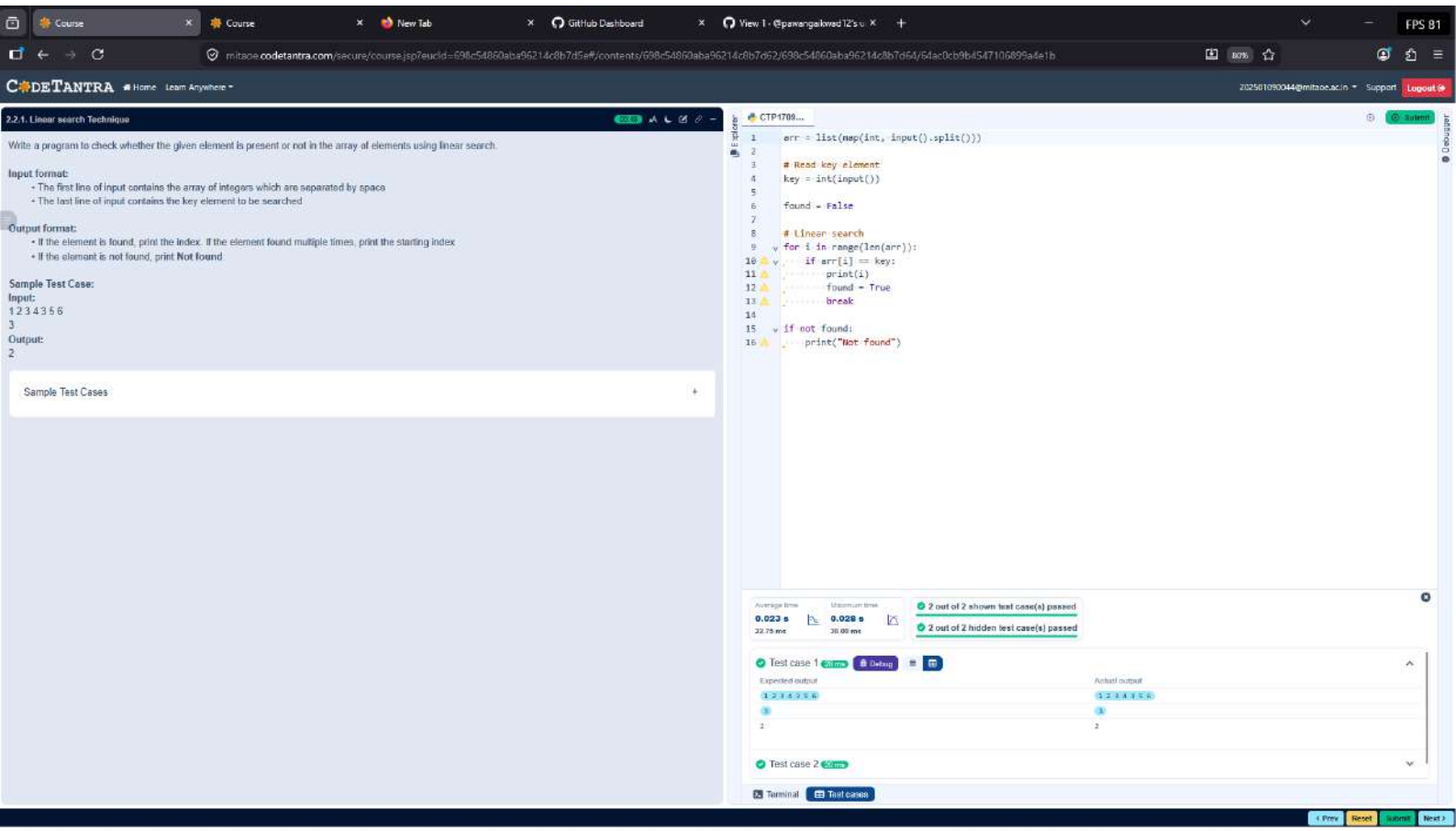

Course
Course
New Tab
GitHub Dashboard
FPS 68
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ec9df4ab787eca9da869
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.2.3. Numpy: Custom Sequence Generation
Write a Python program that takes the following inputs from the user:
• Start value: The starting point of the sequence.
• Stop value: The sequence should end before this value.
• Step value: The increment between each number in the sequence.
The program should then generate a sequence using numpy based on these inputs and print the generated sequence.
Input Format:
• The user will input three integer values: start, stop, and step, each on a new line.
Output Format:
• The program should print the generated sequence based on the input values.
Sample Test Cases
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()# Generate the sequence using np.arange()
# Print the generated sequence
print(np.arange(start,stop,step))
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
[3 5 7 9]
[3 5 7 9]
Terminal
Test cases
Prev
Reset
Submit
Next

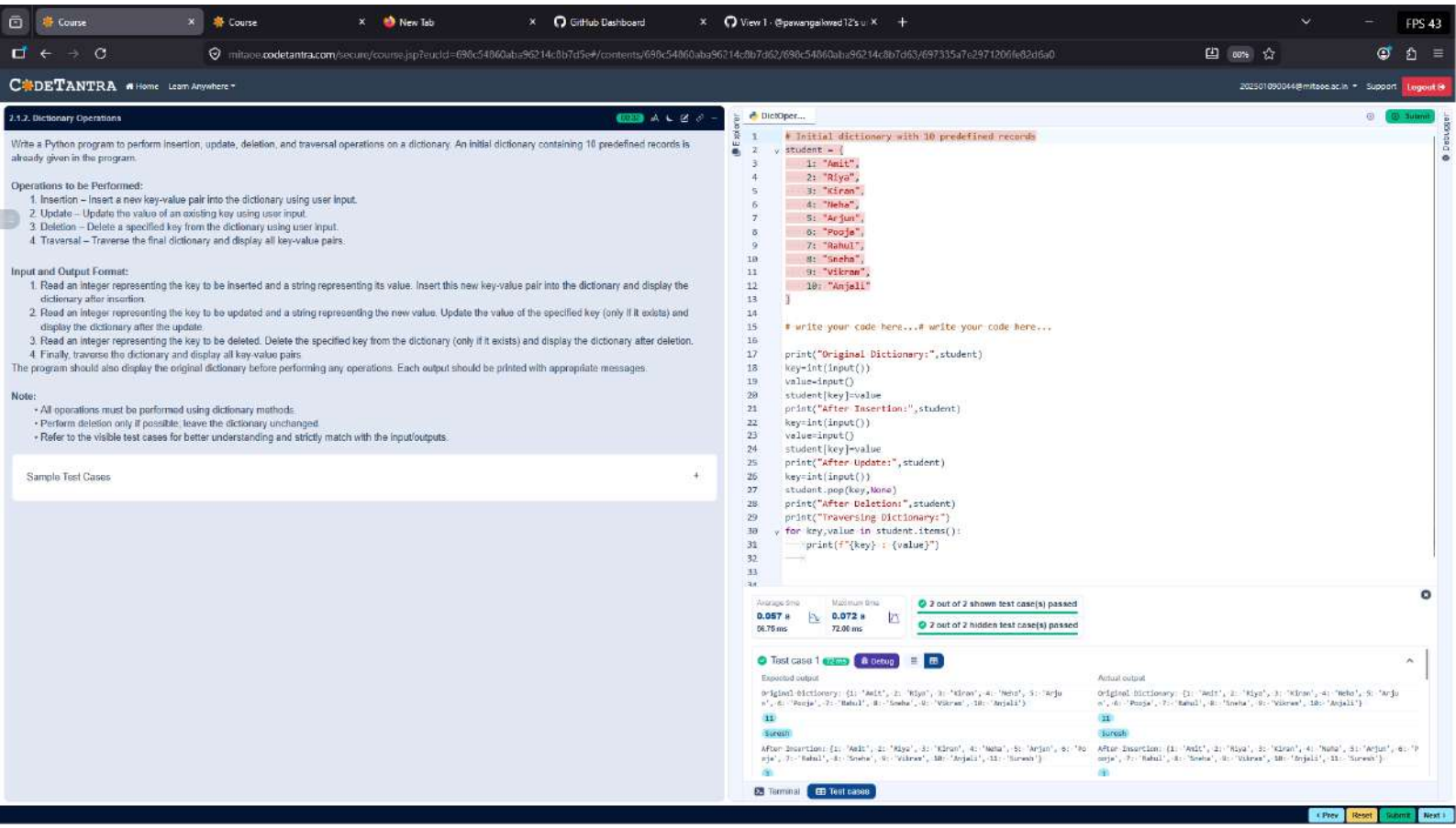

Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 14
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ee19f4ab787eca9da7fe
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.2.2. Numpy: Horizontal and Vertical Stacking of Arrays
You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.
• Horizontal Stacking: Stack the two matrices horizontally (side by side).
• Vertical Stacking: Stack the two matrices vertically (one below the other).
Input Format:
• The program should first prompt the user to input two 3x3 arrays.
• Each array consists of 3 rows, and each row contains 3 space-separated integers.
• The user will input the two arrays row by row.
Output Format:
• The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
• The program should then display the result of the Vertical Stack (one below the other) of the two arrays.
Sample Test Cases
stacking.py
Submit
Explorer
Debugger
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9 # Perform horizontal stacking (hstack)
10
11
12 print("Horizontal Stack:")
13 print(np.hstack((arr1,arr2)))
14 # Perform vertical stacking (vstack)
15 print("Vertical Stack:")
16 print(np.vstack((arr1,arr2)))
17
18
Average time
0.059 s
59.75 ms
Maximum time
0.074 s
74.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
Actual output
Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
Terminal
Test cases
Prev
Reset
Submit
Next

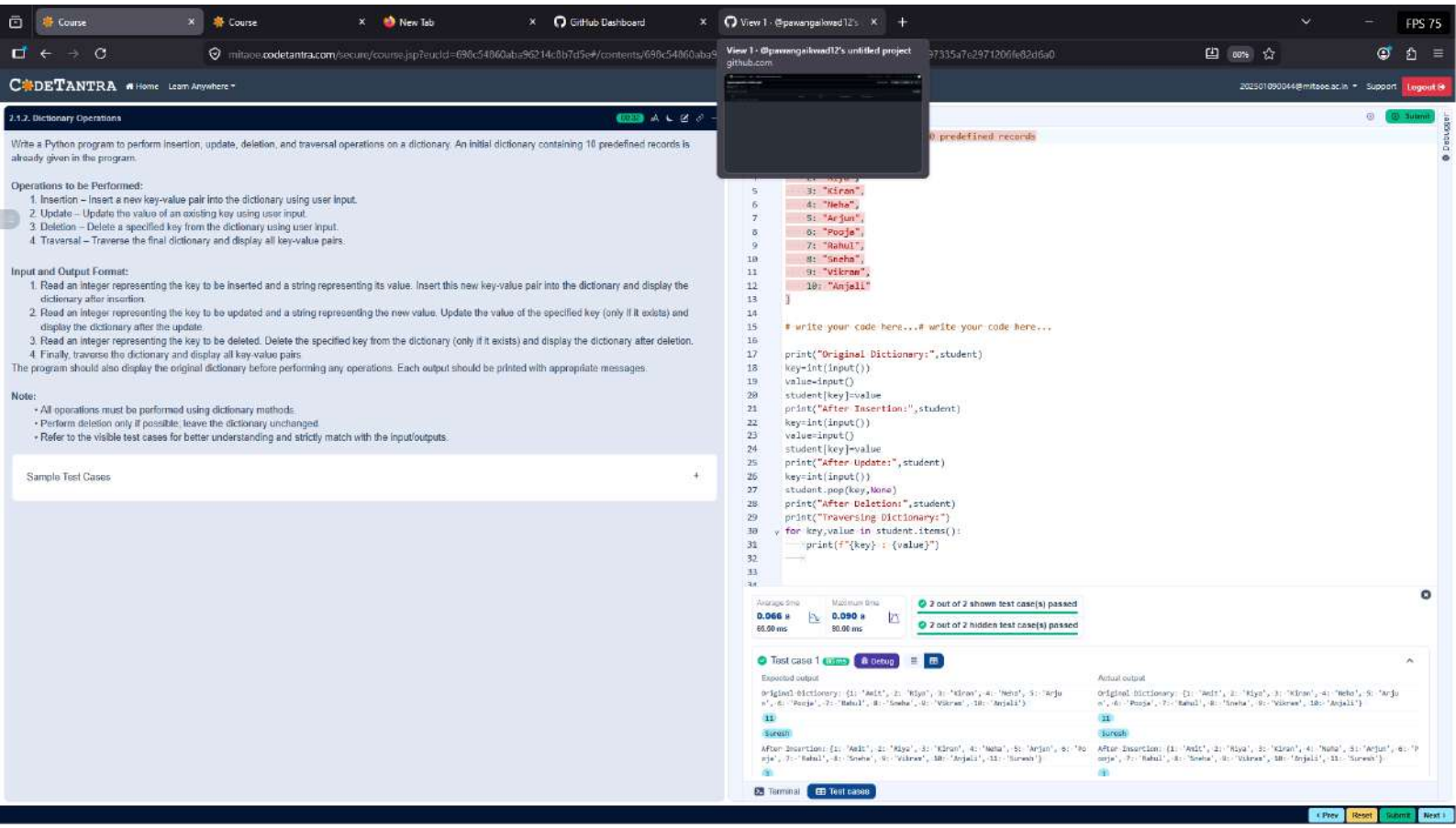

Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 66
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774e3fff4ab787eca9da092
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.2.1. Numpy: Matrix Operations
The given code takes two 3 × 3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.
Task:
You are required to compute and display the results of the following matrix operations:
1. Addition (matrix_a + matrix_b)
2. Subtraction (matrix_a − matrix_b)
3. Element-wise Multiplication (matrix_a * matrix_b)
4. Matrix Multiplication (matrix_a · matrix_b)
5. Transpose of Matrix A
Input Format:
• The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
• Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.
Output Format:
The program should display the results of the operations in the following order:
1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.
Sample Test Cases
matrixOp...
Submit
Explorer
Debugger
import numpy as np
# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
# Addition
print("Addition (A + B):")
print(matrix_a + matrix_b)
# Subtraction
print("Subtraction (A - B):")
print(matrix_a - matrix_b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
print(matrix_a * matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(np.dot(matrix_a,matrix_b))
# Transpose
print("Transpose of A:")
print(np.transpose(matrix_a))
Average time
0.062 s
61.75 ms
Maximum time
0.069 s
69.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
0 ms
Debug
Expected output
Actual output
Enter Matrix A:
1 2 3
4 5 6
7 8 9
Enter Matrix B:
1 1 1
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

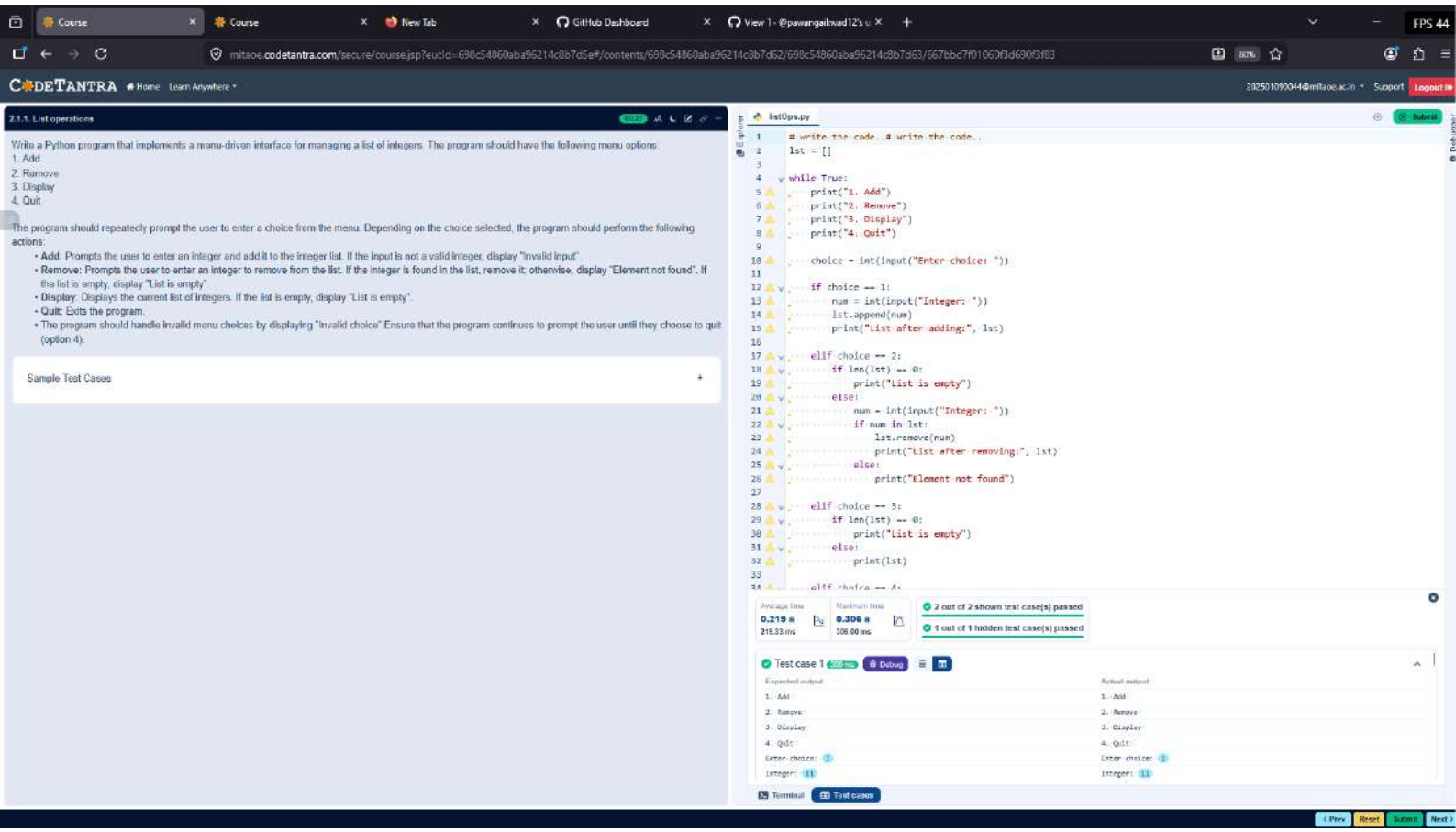

Course
New Tab
GitHub Dashboard
FPS 55
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d66/6684d40e5d953769555a73c1d
CODETANTRA
Home
Learn Anywhere
Support
Logout
3.1.1. Numpy Array Operations
Write a Python program to create a NumPy array based on user input and display the following:
• The created NumPy array
• The number of dimensions of the array using ndim
• The shape of the array using shape
• The total number of elements in the array using size
Assume all input elements are valid integers.
Input Format:
• The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
• The next r lines contain c space-separated integers representing the elements of the array, entered row by row.
Output Format:
• The created NumPy array (printed as a 2D array)
• An integer representing the number of dimensions of the array.
• A tuple representing the shape of the array.
• An integer representing the total number of elements in the array.
Note: Use reshape() function to reshape the input array with the specified number of rows and columns.
Sample Test Cases
numpyarr...
Submit
import numpy as np# write your code here...
rows,columns = map(int,input().split())
elements = []
for i in range(rows):
elements_row = map(int,input().split())
elements.extend(elements_row)
matrix = np.array(elements).reshape(rows,columns)
print(matrix)
print(matrix.ndin)
print(matrix.shape)
print(matrix.size)
Average time
0.036 s
36.17 ms
Maximum time
0.066 s
66.00 ms
2 out of 2 shown test case(s) passed
10 out of 10 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]
[5 6 7 8]]
Terminal
Test cases
Prev
Reset
Submit
Next

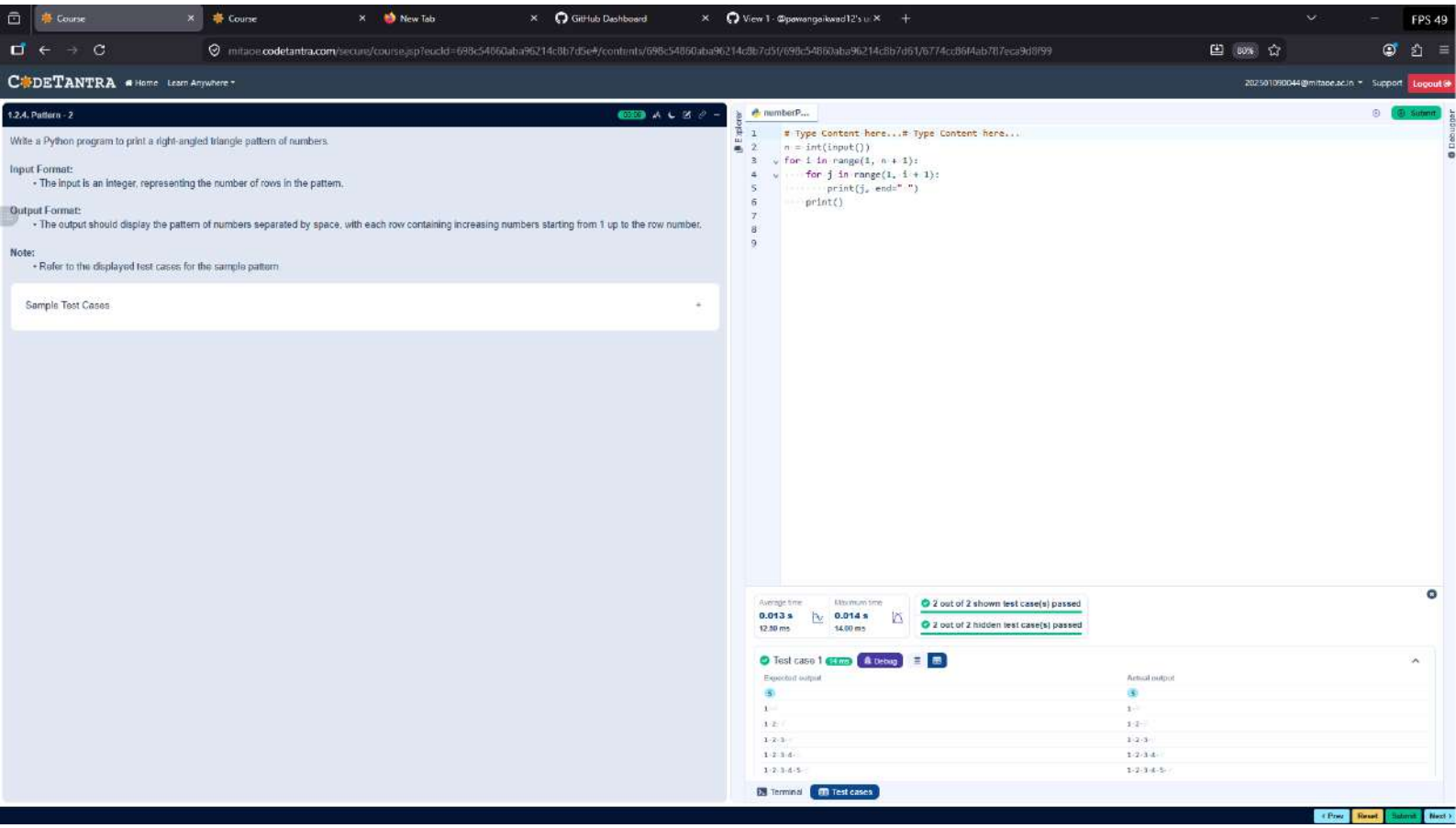

Course
Course
New Tab
GitHub Dashboard
FPS 9
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d64/6774df0abf4ab787eca9d9230
CODETANTRA
Home
Learn Anywhere
Support
Logout
2.2.2. Captain of the Team
You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.
Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.
Output Format
The output should be the height (in centimeters) of the tallest player
Sample Test Cases
captainof...
Submit
heights = list(map(int, input().split()))
tallest = max(heights)
print(tallest)
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
191
Actual output
191
Terminal
Test cases
Prev
Reset
Submit
Next

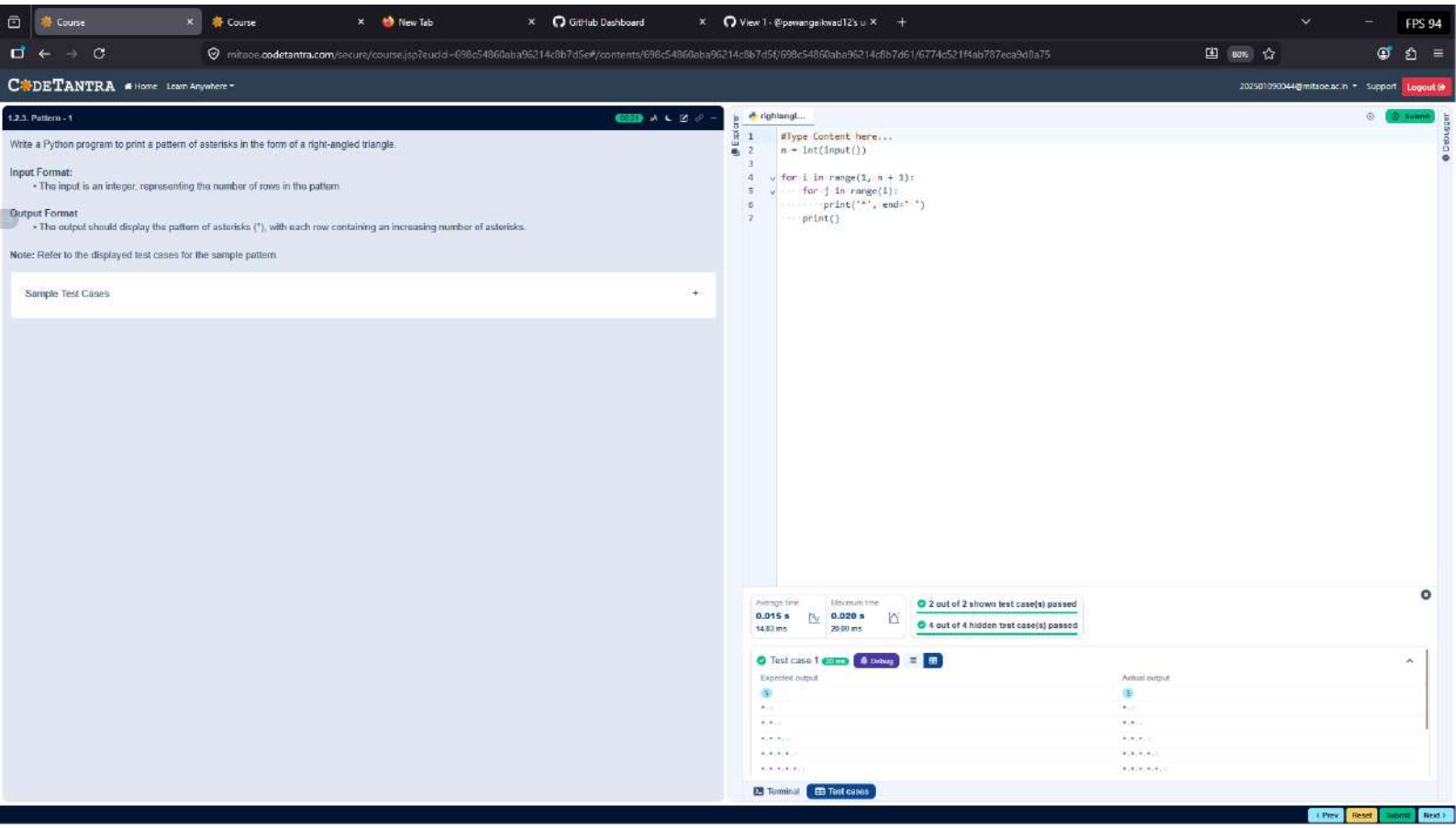

Course
Course
New Tab
GitHub Dashboard
FPS 81
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d62/64ac0cb9b4547106899a4e1b
CODETANTRA
Home
Learn Anywhere
Support
Logout
2.2.1. Linear search Technique
Write a program to check whether the given element is present or not in the array of elements using linear search.
Input format:
• The first line of input contains the array of integers which are separated by space
• The last line of input contains the key element to be searched
Output format:
• If the element is found, print the index. If the element found multiple times, print the starting index
• If the element is not found, print Not found
Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2
Sample Test Cases
CTP1709...
Submit
arr = list(map(int, input().split()))
# Read key element
key = int(input())
found = False
# Linear search
for i in range(len(arr)):
if arr[i] == key:
print(i)
found = True
break
if not found:
print("Not found")
0.023 s
0.028 s
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
Test case 2
Terminal
Test cases
Prev
Reset
Submit
Next

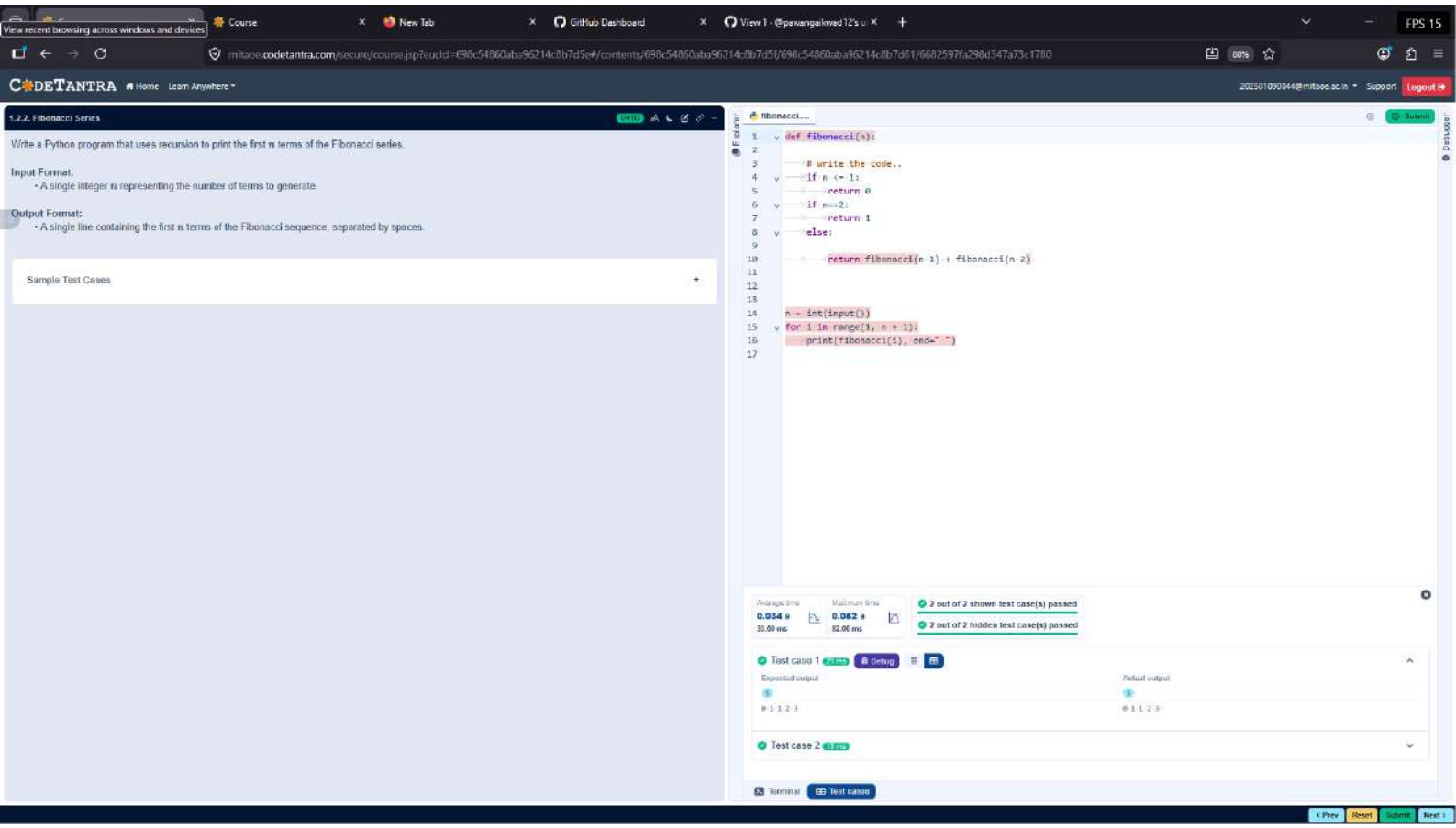

Course
Course
New Tab
GitHub Dashboard
FPS 43
CODETANTRA
Home
Learn Anywhere
Support
Logout
2.1.2. Dictionary Operations
Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.
Operations to be Performed:
1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.
Input and Output Format:
1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.
The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.
Note:
• All operations must be performed using dictionary methods.
• Perform deletion only if possible; leave the dictionary unchanged.
• Refer to the visible test cases for better understanding and strictly match with the input/outputs.
Sample Test Cases
DictOper...
Submit
# Initial dictionary with 10 predefined records
student = {
1: "Amit",
2: "Riya",
3: "Kiran",
4: "Neha",
5: "Arjun",
6: "Pooja",
7: "Rahul",
8: "Sneha",
9: "Vikram",
10: "Anjali"
}
# write your code here...# write your code here...
print("Original Dictionary:",student)
key=int(input())
value=input()
student[key]=value
print("After Insertion:",student)
key=int(input())
value=input()
student[key]=value
print("After Update:",student)
key=int(input())
student.pop(key,None)
print("After Deletion:",student)
print("Traversing Dictionary:")
for key,value in student.items():
print(f"{key} : {value}")
Average time
0.057 s
56.75 ms
Maximum time
0.072 s
72.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
11
Suresh
Terminal
Test cases
Prev
Reset
Submit
Next

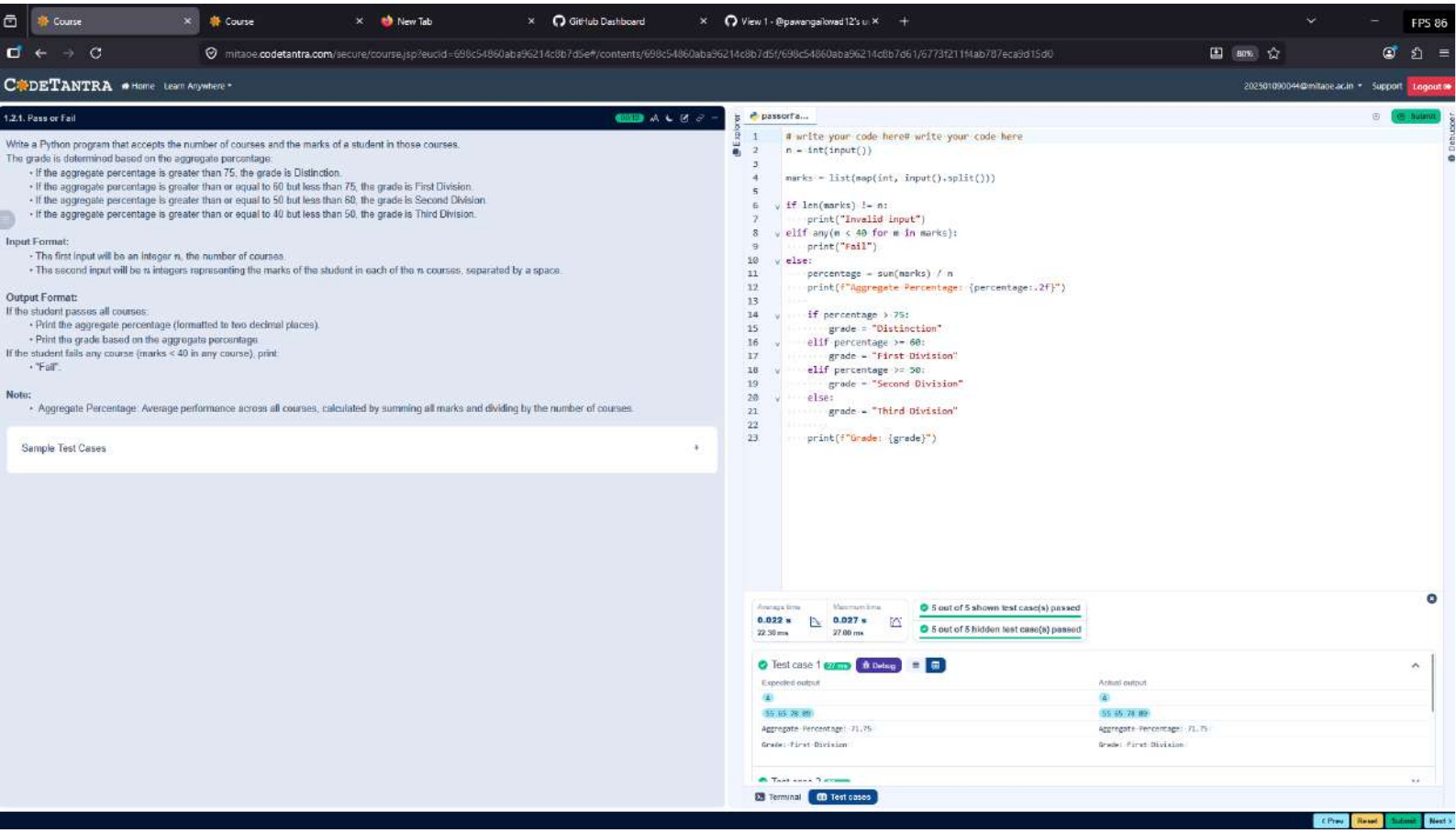

Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's
FPS 75
View 1 · @pawangaikwad12's untitled project
github.com
CODETANTRA
Home
Learn Anywhere
Support
Logout
2.1.2. Dictionary Operations
Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.
Operations to be Performed:
1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.
Input and Output Format:
1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.
The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.
Note:
• All operations must be performed using dictionary methods.
• Perform deletion only if possible; leave the dictionary unchanged.
• Refer to the visible test cases for better understanding and strictly match with the input/outputs.
Sample Test Cases
Submit
predefined records
3: "Kiran",
4: "Neha",
5: "Arjun",
6: "Pooja",
7: "Rahul",
8: "Sneha",
9: "Vikram",
10: "Anjali"
# write your code here...# write your code here...
print("Original Dictionary:",student)
key=int(input())
value=input()
student[key]=value
print("After Insertion:",student)
key=int(input())
value=input()
student[key]=value
print("After Update:",student)
key=int(input())
student.pop(key,None)
print("After Deletion:",student)
print("Traversing Dictionary:")
for key,value in student.items():
print(f"{key} : {value}")
Average time
0.066 s
66.00 ms
Maximum time
0.090 s
90.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
0 ms
Debug
Expected output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
1
Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
1
Terminal
Test cases
Prev
Reset
Submit
Next

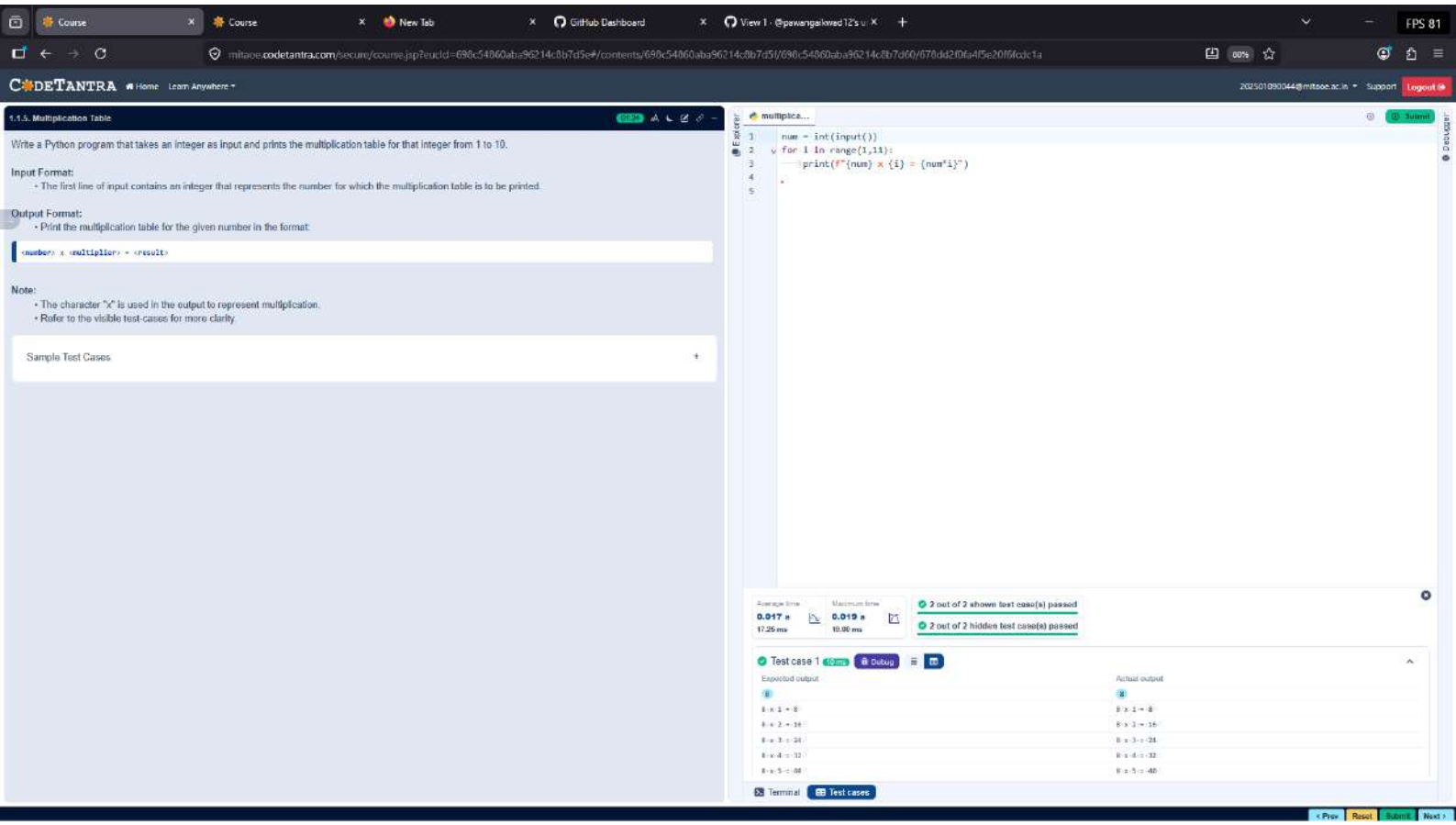

Course
Course
New Tab
GitHub Dashboard
View 1 - @pawangaikwad12's u
FPS 44
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d63/667bbd7f01060f3d690f3f83
CODETANTRA
Home
Learn Anywhere
202501090044@mitaoe.ac.in
Support
Logout
2.1.1. List operations
Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• Display: Displays the current list of integers. If the list is empty, display "List is empty".
• Quit: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).
Sample Test Cases
listOps.py
Submit
# write the code..# write the code..
lst = []
while True:
print("1. Add")
print("2. Remove")
print("3. Display")
print("4. Quit")
choice = int(input("Enter choice: "))
if choice == 1:
num = int(input("Integer: "))
lst.append(num)
print("List after adding:", lst)
elif choice == 2:
if len(lst) == 0:
print("List is empty")
else:
num = int(input("Integer: "))
if num in lst:
lst.remove(num)
print("List after removing:", lst)
else:
print("Element not found")
elif choice == 3:
if len(lst) == 0:
print("List is empty")
else:
print(lst)
Average time
0.219 s
219.33 ms
Maximum time
0.306 s
306.00 ms
2 out of 2 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
Debug
Expected output
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
Actual output
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
Terminal
Test cases
Prev
Reset
Submit
Next

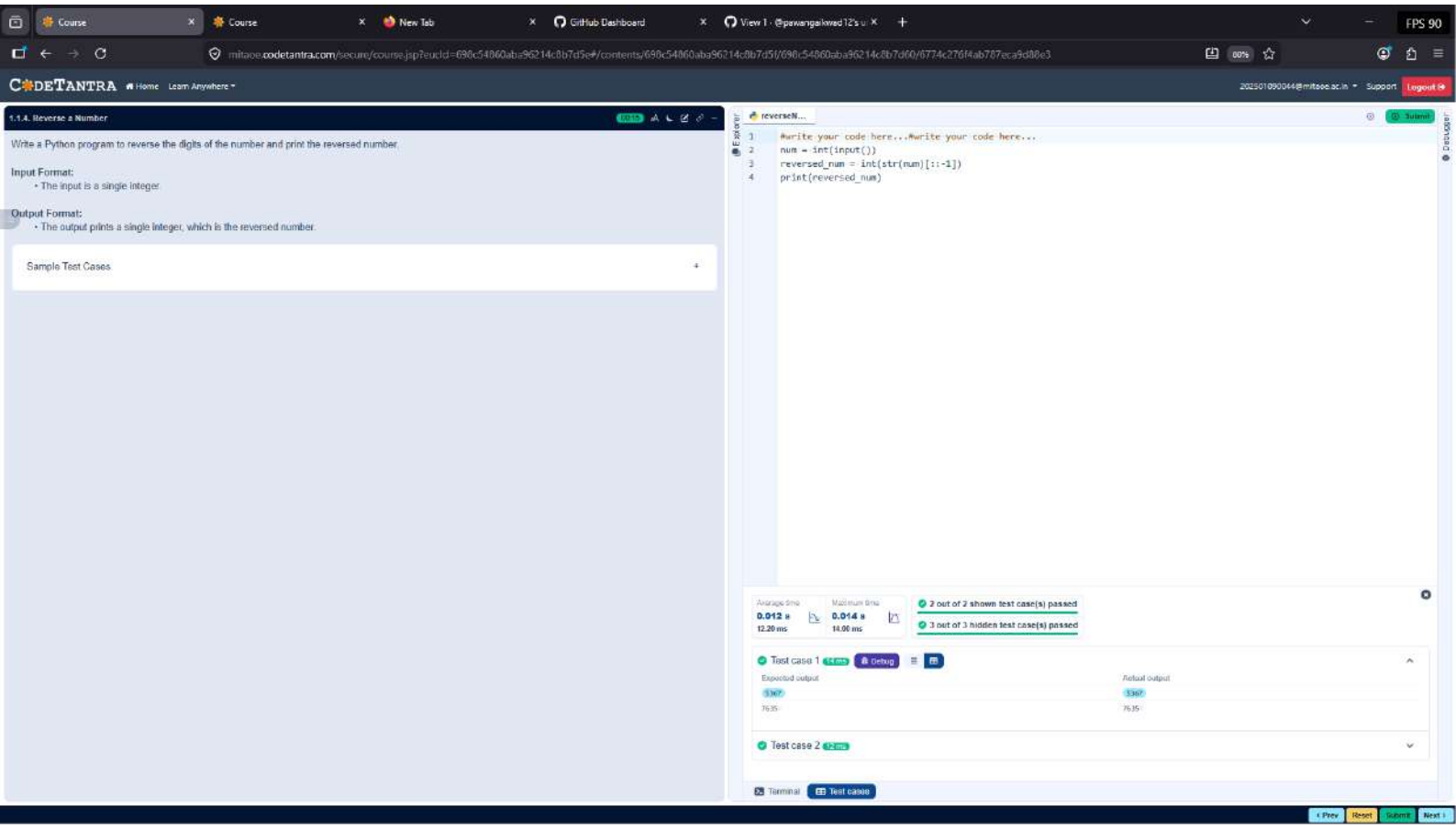

Course
Course
New Tab
GitHub Dashboard
View 1 - @pawangaikwad12's u
FPS 49
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6774cc86f4ab7817eca9d8f99
80%
CODETANTRA
Home
Learn Anywhere
20250109004@mitaoe.ac.in
Support
Logout
1.2.4. Pattern - 2
00:00
Write a Python program to print a right-angled triangle pattern of numbers.
Input Format:
• The input is an integer, representing the number of rows in the pattern.
Output Format:
• The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.
Note:
• Refer to the displayed test cases for the sample pattern
Sample Test Cases
numberP...
Submit
Explorer
Debugger
# Type Content here...# Type Content here...
n = int(input())
for i in range(1, n + 1):
for j in range(1, i + 1):
print(j, end=" ")
print()
Average time
0.013 s
12.50 ms
Minimum time
0.014 s
14.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Terminal
Test cases
Prev
Reset
Submit
Next

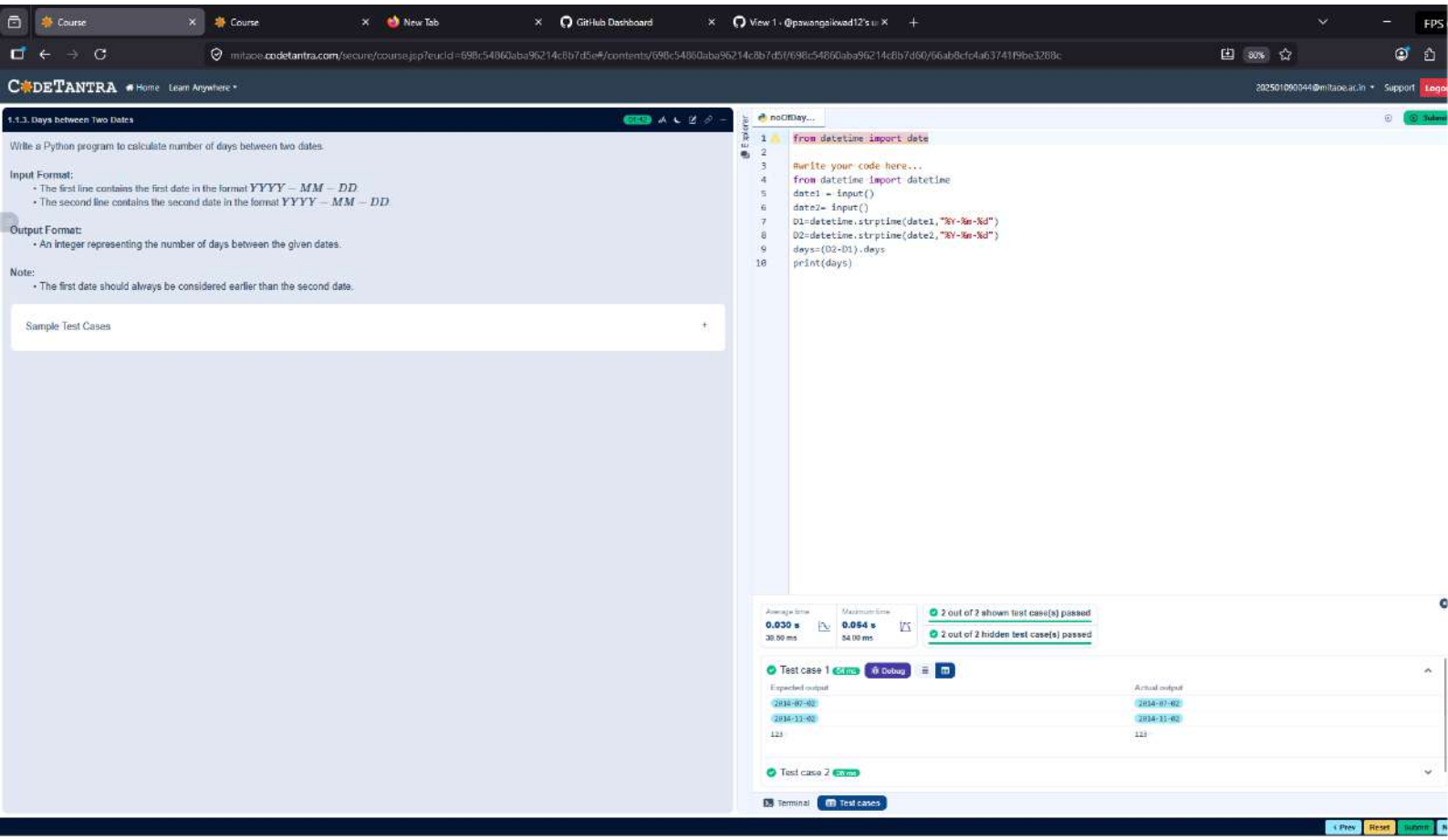

Course
Course
New Tab
GitHub Dashboard
View 1 - @pawangaikwad12's u
FPS 94
mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6774c521f4ab787eca9d8a75
80%
CODETANTRA
Home
Learn Anywhere
20250109044@mitaoe.ac.in
Support
Logout
1.2.3. Pattern - 1
OOPS
Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.
Input Format:
• The input is an integer, representing the number of rows in the pattern.
Output Format
• The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.
Note: Refer to the displayed test cases for the sample pattern.
Sample Test Cases
+
Explorer
rightangl...
Submit
Debugger
1 #Type Content here...
2 n = int(input())
3
4 for i in range(1, n + 1):
5 for j in range(i):
6 print("*", end=" ")
7 print()
Average time
0.015 s
14.83 ms
Maximum time
0.020 s
20.00 ms
2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed
Test case 1
20 ms
Debug
Expected output
Actual output
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›

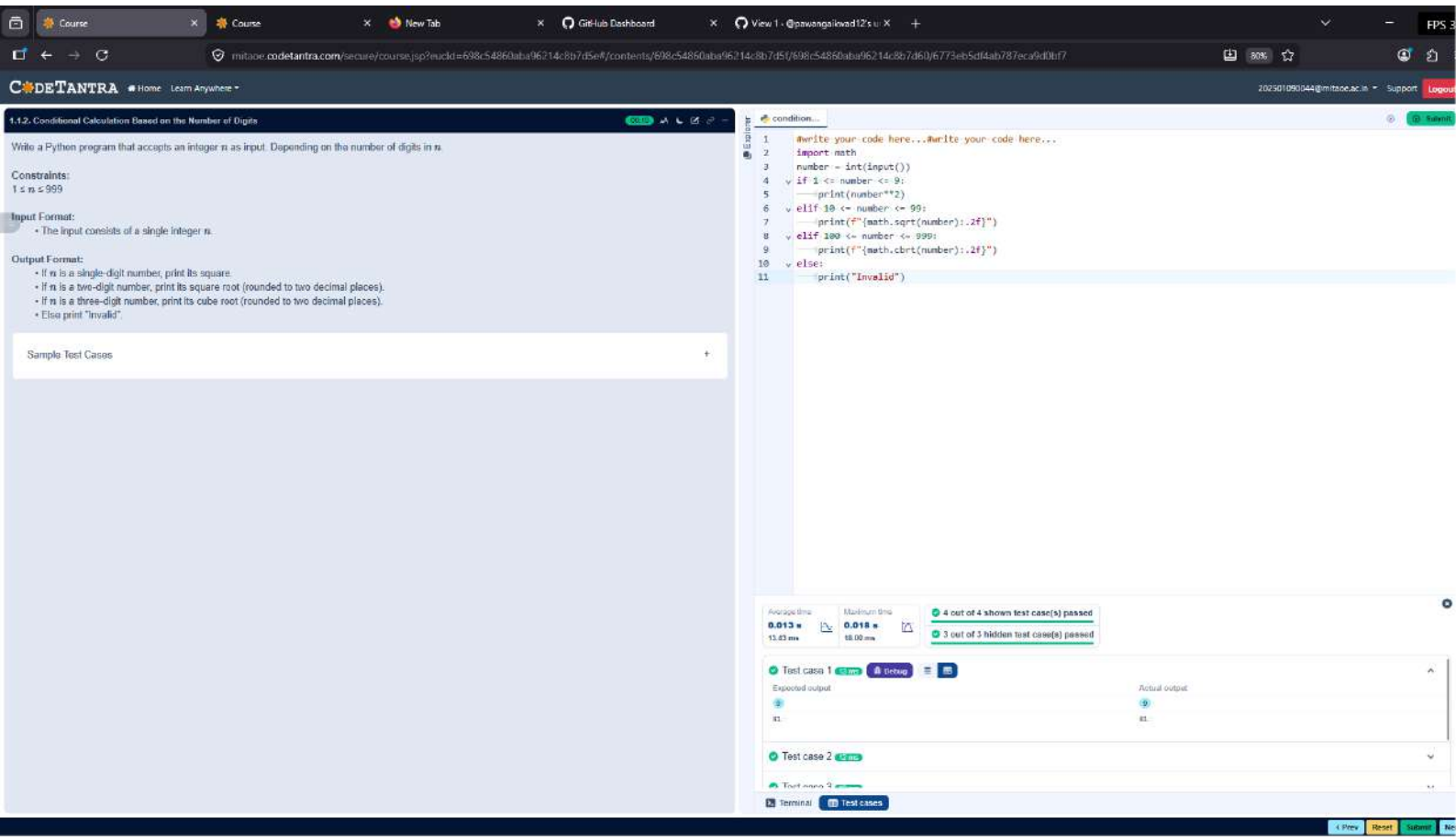

Course
New Tab
GitHub Dashboard
FPS 15
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d51/690c54860aba96214c8b7d61/6682597fa298d347a73c1780
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.2.2. Fibonacci Series
Write a Python program that uses recursion to print the first n terms of the Fibonacci series.
Input Format:
• A single integer n representing the number of terms to generate.
Output Format:
• A single line containing the first n terms of the Fibonacci sequence, separated by spaces.
Sample Test Cases
fibonacci...
Submit
def fibonacci(n):
# write the code..
if n <= 1:
return 0
if n==2:
return 1
else:
return fibonacci(n-1) + fibonacci(n-2)
n = int(input())
for i in range(1, n + 1):
print(fibonacci(i), end=" ")
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
0 1 1 2 3
Test case 2
Terminal
Test cases
Prev
Reset
Submit
Next

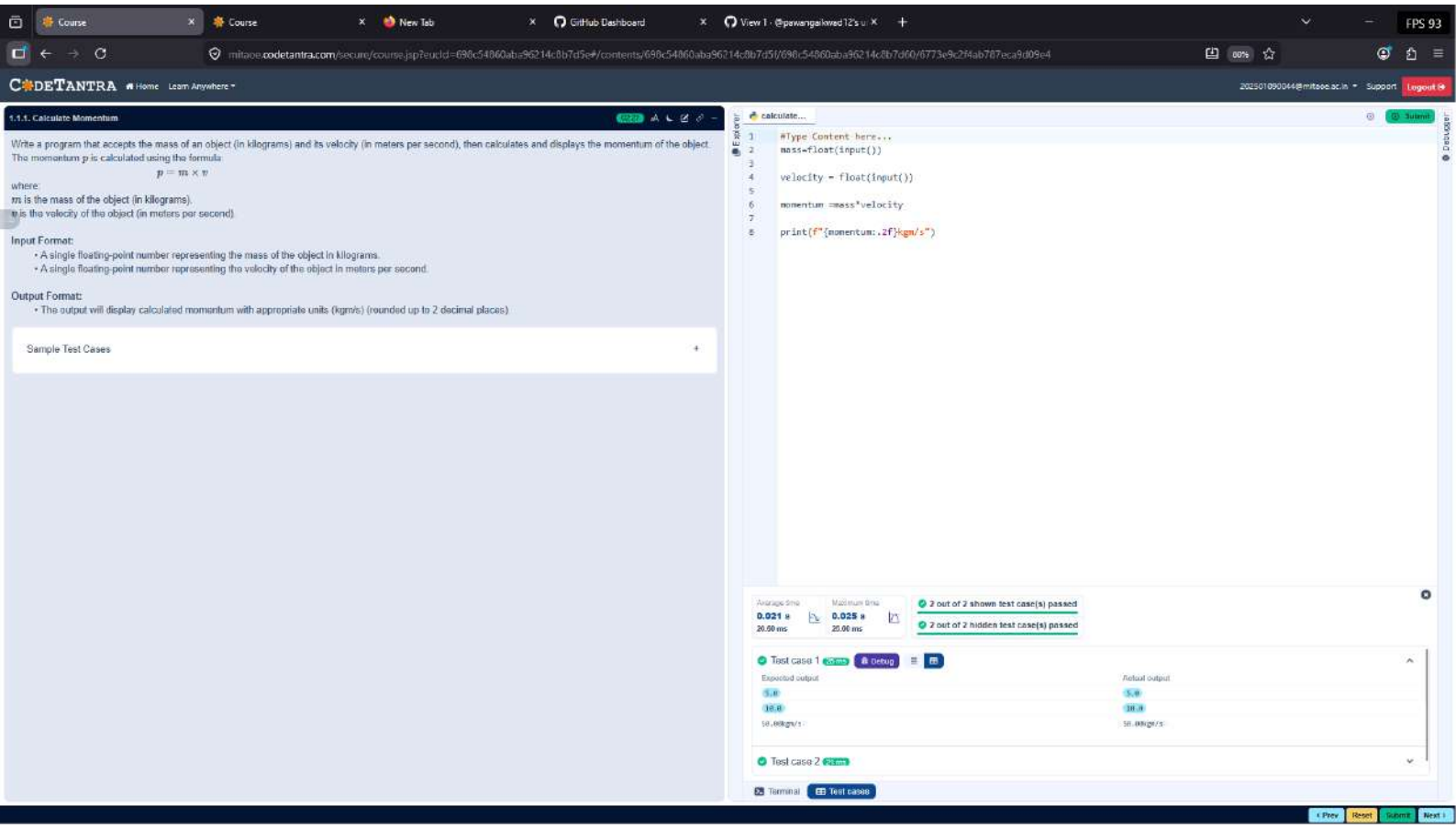

Course
Course
New Tab
GitHub Dashboard
FPS 86
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6773f211f4ab787eca9d15d0
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.2.1. Pass or Fail
Write a Python program that accepts the number of courses and the marks of a student in those courses.
The grade is determined based on the aggregate percentage:
• If the aggregate percentage is greater than 75, the grade is Distinction.
• If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
• If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
• If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.
Input Format:
• The first input will be an integer n, the number of courses.
• The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.
Output Format:
If the student passes all courses:
• Print the aggregate percentage (formatted to two decimal places).
• Print the grade based on the aggregate percentage.
If the student fails any course (marks < 40 in any course), print:
• "Fail".
Note:
• Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.
Sample Test Cases
passorfa...
Submit
# write your code here# write your code here
n = int(input())
marks = list(map(int, input().split()))
if len(marks) != n:
print("Invalid input")
elif any(m < 40 for m in marks):
print("Fail")
else:
percentage = sum(marks) / n
print(f"Aggregate Percentage: {percentage:.2f}")
if percentage > 75:
grade = "Distinction"
elif percentage >= 60:
grade = "First Division"
elif percentage >= 50:
grade = "Second Division"
else:
grade = "Third Division"
print(f"Grade: {grade}")
5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
Aggregate Percentage: 71.75
Grade: First Division
Terminal
Test cases
Prev
Reset
Submit
Next


Course
Course
New Tab
GitHub Dashboard
FPS 81
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d50/696c54860aba96214c8b7d60/670dd2f0fa4f5e20f6fcdc1a
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.1.5. Multiplication Table
Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.
Input Format:
• The first line of input contains an integer that represents the number for which the multiplication table is to be printed.
Output Format:
• Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>
Note:
• The character "x" is used in the output to represent multiplication.
• Refer to the visible test cases for more clarity.
Sample Test Cases
Explorer
multiplica...
Submit
Debugger
num = int(input())
for i in range(1,11):
print(f"{num} x {i} = {num*i}")
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
Terminal
Test cases
Prev
Reset
Submit
Next


Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 90
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d5f/690c54860aba96214c8b7d60/6774c276f4ab787eca9d88e3
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.1.4. Reverse a Number
Write a Python program to reverse the digits of the number and print the reversed number.
Input Format:
• The input is a single integer.
Output Format:
• The output prints a single integer, which is the reversed number.
Sample Test Cases
reverseN...
Submit
#write your code here...#write your code here...
num = int(input())
reversed_num = int(str(num)[::-1])
print(reversed_num)
Average time
0.012 s
12.20 ms
Maximum time
0.014 s
14.00 ms
2 out of 2 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
Debug
Expected output
5367
7635
Actual output
5367
7635
Test case 2
Terminal
Test cases
‹ Prev
Reset
Submit
Next ›


Course
Course
New Tab
GitHub Dashboard
mitaoe.codetantra.com/secure/courseJsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d60/66ab8cfc4a63741f9be3288c
CODETANTRA
Home
Learn Anywhere
1.1.3. Days between Two Dates
Write a Python program to calculate number of days between two dates.
Input Format:
• The first line contains the first date in the format YYYY − MM − DD.
• The second line contains the second date in the format YYYY − MM − DD.
Output Format:
• An integer representing the number of days between the given dates.
Note:
• The first date should always be considered earlier than the second date.
Sample Test Cases
noOfDay...
from datetime import date
#write your code here...
from datetime import datetime
date1 = input()
date2= input()
D1=datetime.strptime(date1,"%Y-%m-%d")
D2=datetime.strptime(date2,"%Y-%m-%d")
days=(D2-D1).days
print(days)
0.030 s
38.50 ms
0.054 s
54.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
2014-07-02
2014-11-02
123
Test case 2
Terminal
Test cases
Prev
Reset
Submit


Course
Course
New Tab
GitHub Dashboard
mitaoe.codetantra.com/secure/courseJsp?eucId=698c54860aba96214c8b7d5f#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d60/6773eb5df4ab787eca9d0bf7
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.1.2. Conditional Calculation Based on the Number of Digits
Write a Python program that accepts an integer n as input. Depending on the number of digits in n.
Constraints:
1 ≤ n ≤ 999
Input Format:
• The input consists of a single integer n.
Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".
Sample Test Cases
condition...
Submit
#write your code here...#write your code here...
import math
number = int(input())
if 1 <= number <= 9:
print(number**2)
elif 10 <= number <= 99:
print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
print(f"{math.cbrt(number):.2f}")
else:
print("Invalid")
4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
Debug
Expected output
Actual output
Test case 2
Terminal
Test cases
Prev
Reset
Submit


Course
Course
New Tab
GitHub Dashboard
View 1 · @pawangaikwad12's u
FPS 93
mitaoe.codetantra.com/secure/course.jsp?eucId=690c54860aba96214c8b7d5e#/contents/690c54860aba96214c8b7d5f/690c54860aba96214c8b7d60/6773e9c2f4ab787eca9d09e4
CODETANTRA
Home
Learn Anywhere
Support
Logout
1.1.1. Calculate Momentum
Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object.
The momentum p is calculated using the formula:
$$p = m \times v$$
where:
m is the mass of the object (in kilograms).
v is the velocity of the object (in meters per second).
Input Format:
• A single floating-point number representing the mass of the object in kilograms.
• A single floating-point number representing the velocity of the object in meters per second.
Output Format:
• The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places)
Sample Test Cases
calculate...
Submit
Explorer
Debugger
#Type Content here...
mass=float(input())
velocity = float(input())
momentum =mass*velocity
print(f"{momentum:.2f}kgm/s")
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
Debug
Test case 2
Terminal
Test cases
Prev
Reset
Submit
Next